Vel Tech
Rangarajan Dr. Sagunthala
R&D Institute of Science and Technology
DEEMED TO BE
University
(Estd. u/s 3 of UGC Act, 1956)
Avadi, Chennai

School of Computing

Department of Computer Science and Engineering

ACADEMIC YEAR: 2025-26

(WINTER SEMESTER)

LAB RECORD

10211CS224 - Full Stack Application Development

NAME : Chilukuri Vennela Reddy

VTU.NO : VTU25619

REG.NO : 23UECS0956

BRANCH : CSE

YEAR/SEM : 3rd/6th

SLOT : E2

10211CS224 - Full Stack Application Development

TABLE OF CONTENTS

Task 1: Student Registration & Data Storage

Design a Student Registration Form using HTML5 and CSS3 that collects: Name, Email, DOB, Department, Phone Store the data in a database table and retrieve it using SELECT.

AIM:

To design a Student Registration Form using HTML5 and CSS3 that collects student details - Name, Email, Date of Birth, Department, and Phone Number and to store the submitted data into a relational database table and retrieve it using SELECT statements.

PROCEDURE:

Step 1: Create an HTML5 file with a<form>element containing fields: Name, Email, DOB, Department, Phone.

Step 2: Apply HTML5 validation attributes: required,type = "email",type = "date",patternfor phone.

Step 3: Style the form using CSS3: useflexbox/gridlayout, apply colours, borders, hover effects, and a submit button.

Step 4: Create a MySQL/SQLite database and switch to it usingCREATE DATABASEandUSE.

Step 5: Create astudentstable with appropriate data types for each field.

Step 6: Insert sample student records usingINSERT INTOstatements.

Step 7: Retrieve all records usingSELECT * FROM students;and verify the output.

Step 8: Apply optional filters usingWHERE,ORDER BY, andLIMITclauses.

PROGRAM:

Student registration.HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Student Registration</title>
<style>
body {
  font-family: Arial;
  background: #eef2f7;
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
}
.container
  { background: #fff;
  padding: 20px;
  width: 350px;
  border-radius: 10px;
  box-shadow: 0 5px 15px rgba(0,0,0,0.2);
}
h2 { text-align: center; }
input, select, button
{ width: 100%;
  margin: 8px 0;
  padding: 8px;
}
button {
  background: #1a73e8;
```

```
  color: white;
  border: none;
  cursor: pointer;
}
table {
  width: 100%;
  margin-top: 10px;
  border-collapse: collapse;
}
td, th {
  border: 1px solid #ccc;
  padding: 5px;
}
</style>
</head>
<body>
<div class="container">
<h2>Student Registration</h2>
<form id="form">
  <input type="text" id="name" placeholder="Full Name" required>
  <input type="email" id="email" placeholder="Email" required>
  <input type="date" id="dob" required>
  <input type="tel" id="phone" placeholder="Phone (10 digits)" required>
  <select id="dept" required>
    <option value="">Select Department</option>
    <option>Computer Science</option>
    <option>IT</option>
    <option>ECE</option>
  </select>
  <button type="submit">Register</button>
  <button type="reset">Clear</button>
</form>
```

```
<p id="msg" style="color:green;"></p>
<table id="table" style="display:none;">
<thead><tr><th>Field</th><th>Value</th></tr></thead>
<tbody id="data"></tbody>
</table>
</div>
<script>
const form = document.getElementById("form");
form.onsubmit = e => {
  e.preventDefault();
  const name = nameInput.value = document.getElementById("name").value;
  const email = document.getElementById("email").value;
  const dob = document.getElementById("dob").value;
  const phone = document.getElementById("phone").value;
  const dept = document.getElementById("dept").value;
  if (phone.length != 10 || isNaN(phone))
    { alert("Invalid phone number");
    return;
  }
  document.getElementById("msg").innerText = "Registration Successful!";
  const fields = {
    ID: Math.floor(Math.random()*1000),
    Name: name,
    Email: email,
    DOB: dob,
    Dept: dept,
    Phone: phone
  };
  const tbody = document.getElementById("data");
  tbody.innerHTML = "";
  for (let key in fields) {
    tbody.innerHTML += `<tr><td>${key}</td><td>${fields[key]}</td></tr>`;
```

```
  }
  document.getElementById("table").style.display = "table";
};
form.onreset = () =>
  { document.getElementById("msg").innerText = "";
  document.getElementById("table").style.display = "none";
};
</script>
</body>
</html>
```

SQL

STEP 1: Create and select the database

```
CREATE DATABASE IF NOT EXISTS college_db;
USE college_db;
```

STEP 2: Create the students table

```
CREATE TABLE IF NOT EXISTS students (
    student_id  INT AUTO_INCREMENT PRIMARY KEY,
    full_name    VARCHAR(100)  NOT NULL,
    email        VARCHAR(100)  NOT NULL UNIQUE,
    dob          DATE          NOT NULL,
    department   VARCHAR(80)   NOT NULL,
    phone        CHAR(10)      NOT NULL,
    created_at   TIMESTAMP     DEFAULT CURRENT_TIMESTAMP
);
```

STEP 3: Insert sample student records

```
INSERT INTO students (full_name, email, dob, department, phone) VALUES
    ('Arun Kumar',      'arun@student.edu',     '2003-05-14', 'Computer Science',
'9876543210'),
    ('Priya Sharma',    'priya@student.edu',    '2002-11-22', 'Electronics and Communication',
'8765432109'),
    ('Ravi Menon',      'ravi@student.edu',     '2003-08-01', 'Mechanical Engineering',
'7654321098'),
```

('Sneha Iyer', 'sneha@student.edu', '2004-01-30', 'Electrical Engineering', '6543210987'),

('Karthik Raja', 'karthik@student.edu', '2002-07-19', 'Computer Science', '9123456780'),

('Divya Nair', 'divya@student.edu', '2003-03-11', 'Information Technology', '9988776655'),

('Mohammed Arif', 'arif@student.edu', '2002-09-05', 'Civil Engineering', '9871234560');

STEP 4: SELECT Queries - Retrieve Data

4a. Retrieve ALL student records

```
SELECT * FROM students;
```

4b. Retrieve specific columns only

```
SELECT student_id, full_name, email, department
FROM students;
```

4c. Filter by department

```
SELECT full_name, email, phone
FROM students
WHERE department = 'Computer Science';
```

4d. Sort students by name (A to Z)

```
SELECT full_name, department, dob
FROM students
ORDER BY full_name ASC;
```

4e. Count total students per department

```
SELECT department, COUNT(*) AS total_students
FROM students
GROUP BY department
ORDER BY total_students DESC;
```

4f. Find students born after 2003-01-01

```
SELECT full_name, dob, department
FROM students
WHERE dob > '2003-01-01'
ORDER BY dob;
```

4g. Search student by email

SELECT * FROM students

WHERE email = 'arun@student.edu';

4h. Limit output to first 3 records

SELECT * FROM students LIMIT 3;

STEP 5: UPDATE a record

UPDATE students

SET phone = '9000011111'

WHERE email = 'arun@student.edu';

Verify update

SELECT full_name, phone FROM students WHERE email = 'arun@student.edu';

STEP 6: DELETE a record

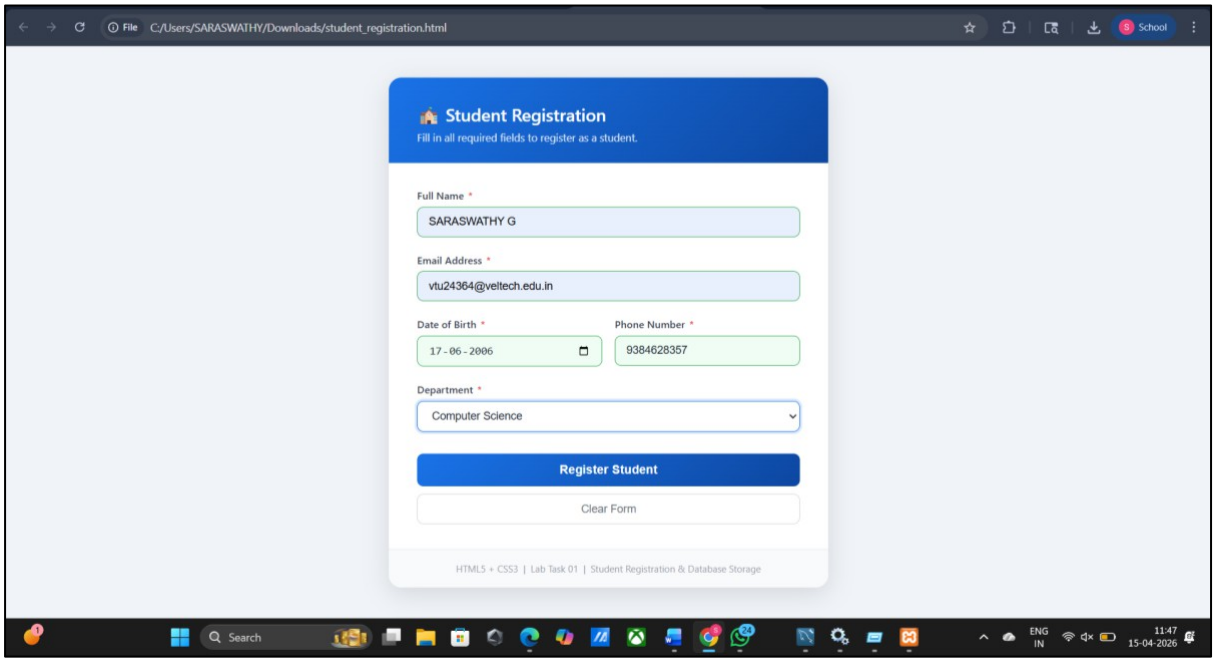
DELETE FROM students WHERE student_id = 7;

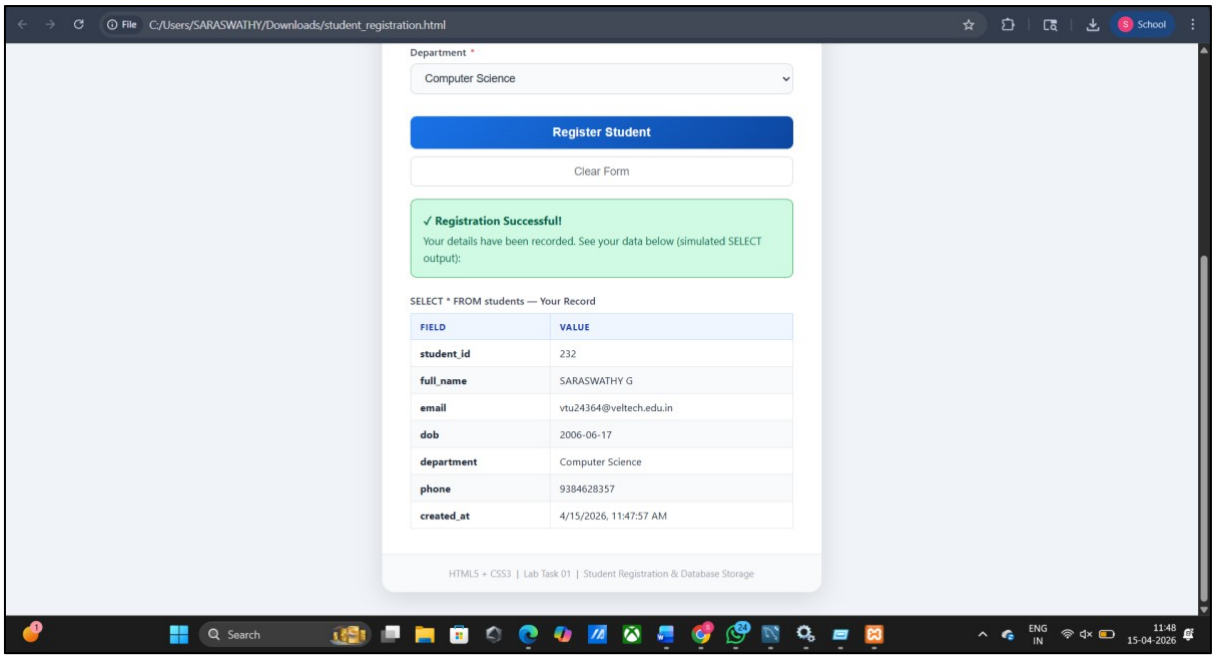
(Commented out to preserve data — uncomment to run)

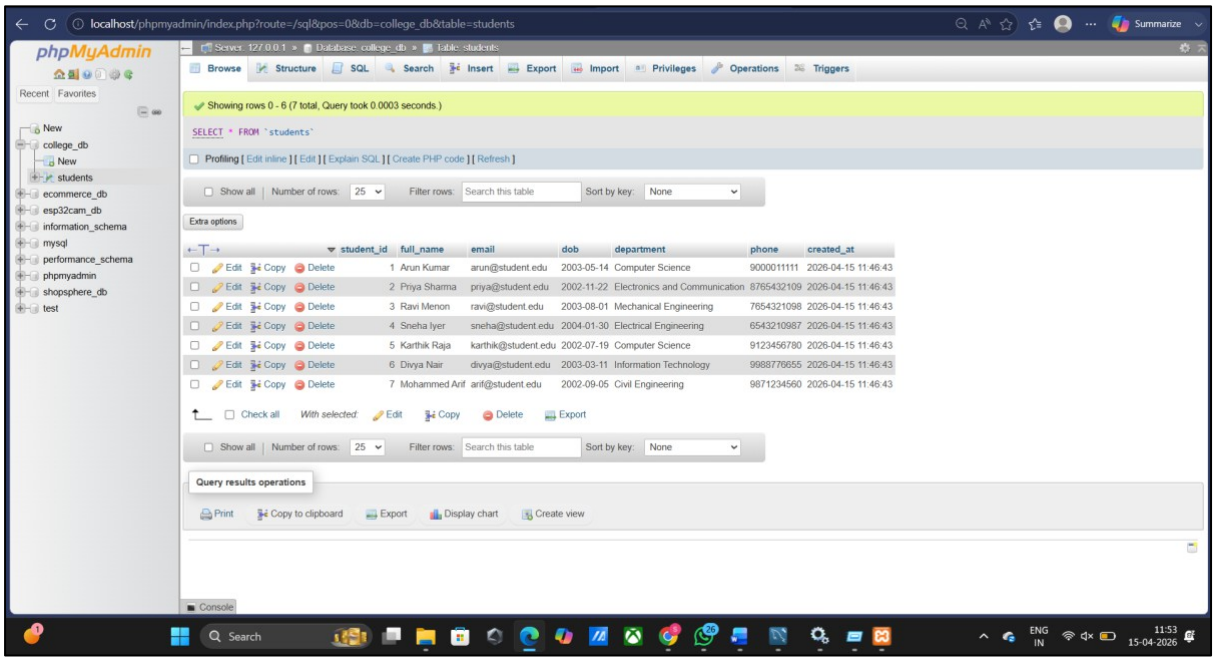
STEP 7: Describe table structure

DESCRIBE students;

OUTPUT:







RESULT:

The Student Registration Form was created using HTML5 and CSS3 with proper validation. A MySQL table students was designed with fields like student_id, full_name, email, dob, department, phone, and created_at. Data was inserted using INSERT and retrieved using SELECT, with filtering and grouping done using WHERE and GROUP BY, confirming successful database operations.

Task 2: Data Retrieval & Sorting Dashboard

1. Aim: To develop a dynamic web-based dashboard capable of retrieving, filtering, and sorting student/employee records. The system aims to provide real-time data manipulation and categorical analysis (counting) for efficient record management.

2. Algorithm Data Definition:

Initialize an array of objects representing records with keys: Name, Department, and Date. Input Capture: Listen for change events on 'Sort' and 'Filter' dropdown menus.

Filtering Logic: Use the filter() method to create a subset of data matching the selected department.

Sorting Logic: Apply sort() based on the chosen criteria (Alphabetical for Name, Chronological for Date).

Aggregation: Use reduce() or a loop to calculate the frequency of each department in the original dataset.

DOM Manipulation: Dynamically clear the existing table and inject new rows using the processed data.

```
<!DOCTYPE html>
<html>
<head>
    <title>Data Dashboard</title>
    <style>
        body { font-family: Arial; padding: 20px; }
        .stats { font-weight: bold; color: #2c3e50; margin-top: 10px; } table
        { width: 100%; border-collapse: collapse; margin-top: 10px; }
          th, td { border: 1px solid #ccc; padding: 8px; text-align: left; }
    </style>
</head>
<body>
```

Result:Successfully develop a login system that validates user inputs on the client side, checks the entered username and password with stored credentials, and displays appropriate success or error messages dynamically.

```
    <h3>Record Management</h3>
```

```
<select id="sort" onchange="updateView()">
    <option value="name">Sort by Name</option>
    <option value="date">Sort by Date</option>
</select>
<select id="filter" onchange="updateView()">
    <option value="All">All Depts</option>
    <option value="IT">IT</option>
    <option value="Sales">Sales</option>
</select>

<div id="counts" class="stats"></div>

<table id="table">
    <thead><tr><th>Name</th><th>Dept</th><th>Date</th></tr></thead>
    <tbody id="tbody"></tbody>
</table>

<script>
    const data = [
        {n:"John", d:"IT", dt:"2023-01-01"},
        {n:"Sara", d:"Sales", dt:"2022-12-01"},
        {n:"Alex", d:"IT", dt:"2023-05-10"}
    ];

    function updateView() {
        let sortVal = document.getElementById('sort').value;
        let filterVal = document.getElementById('filter').value;

        let filtered = data.filter(i => filterVal === "All" || i.d === filterVal);
        filtered.sort((a,b) => sortVal === "name" ?
            a.n.localeCompare(b.n) : new Date(b.dt) - new Date(a.dt));

        document.getElementById('tbody').innerHTML = filtered.map(i =>
            `<tr><td>${i.n}</td><td>${i.d}</td><td>${i.dt}</td></tr>`
```

```
        ).join('');

        const counts = data.reduce((acc, i) =>
            { acc[i.d] = (acc[i.d] || 0) + 1; return acc;
        }, {});
        document.getElementById('counts').innerText =
            `IT: ${counts['IT'] || 0} | Sales: ${counts['Sales'] || 0}`;
    }
    UpdateView();
    </script>
    </body>
    </html>
```

4. Outputs

The dashboard renders an interactive table. Users can observe:

- **Sorting:** Clicking "Sort by Date" moves the most recent entry to the top.
- **Filtering:** Selecting "IT" removes Sales staff from the view.
- **Counting:** A summary line consistently shows "IT: 2 | Sales: 1" regardless of filters.

3.Login System with Input Validation and Authentication

Aim:

To develop a login system that validates user inputs on the client side, checks the entered username and password with stored credentials, and displays appropriate success or error messages dynamically.

Algorithm:

1. Start
2. Create and display a login form with username and password input fields
3. Wait for the user to click the login button
4. Read the entered username and password values
5. Remove any extra spaces from the inputs
6. Check if the username or password field is empty
7. Validate the input length (minimum required characters)
   If the condition is not satisfied, display a validation message
8. Compare the entered credentials with stored username and password
9. If both match, display "Login successful else, display "Invalid username or password"
10. Stop

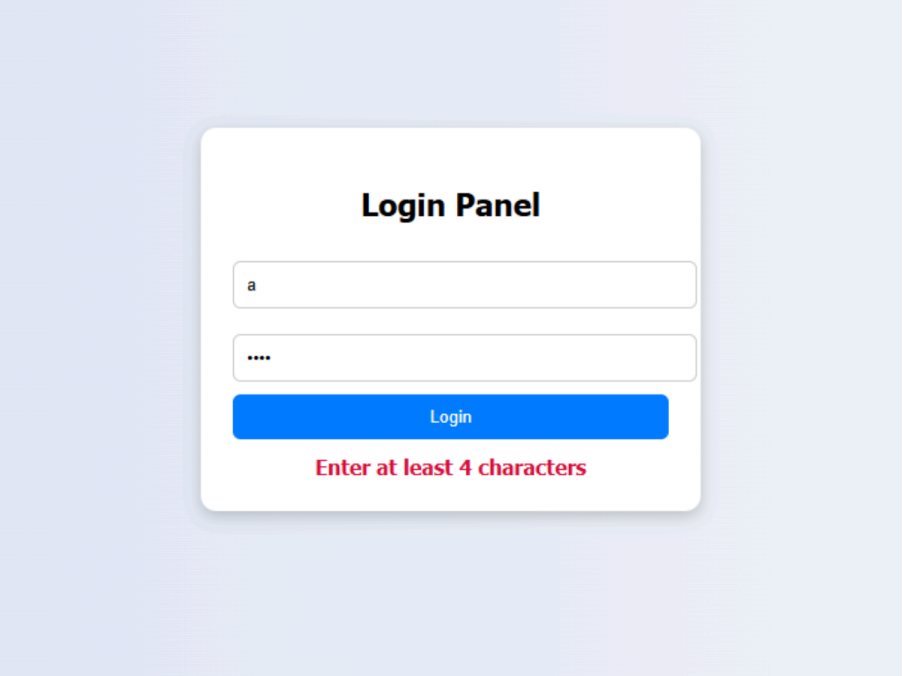
Program:

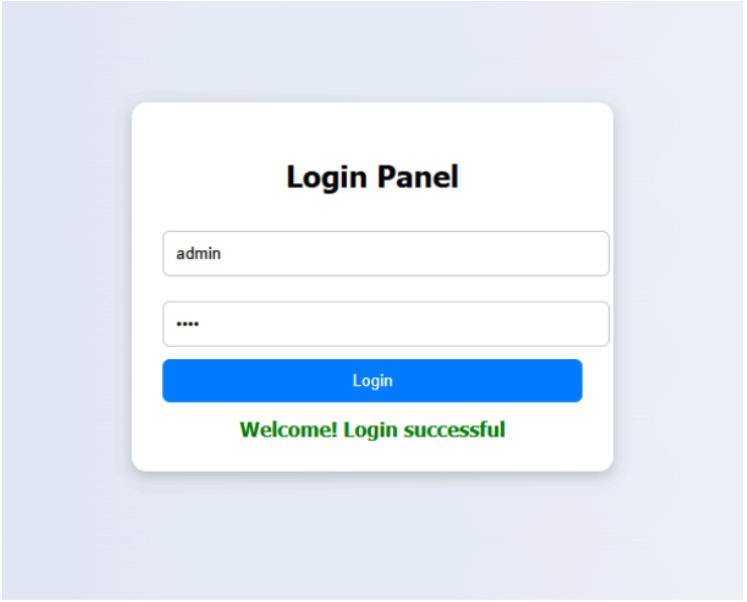
```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Secure Login</title>
    <style>
        body {
            margin: 0;
            font-family: Tahoma;
            background: linear-gradient(to right, #d9e4f5, #f7f7f7);
        }
        .box {
            width: 340px;
            margin: 120px auto;
            padding: 25px;
            background: #ffffff;
            border-radius: 12px;
            box-shadow: 0 4px 12px rgba(0,0,0,0.2);
        }
        h2 {
            text-align: center;
        }
        .input-field {
            width: 100%;
            padding: 10px;
            margin: 10px 0;
```

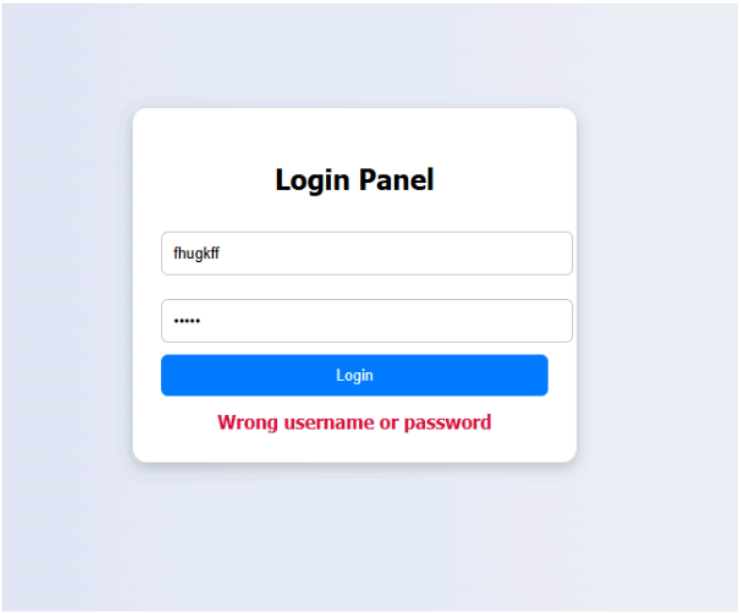
```
            border: 1px solid #ccc;
            border-radius: 6px;
        }
        .btn {
    width: 100%;
            padding: 10px;
            background: #007bff;
            color: white;
            border: none;
            border-radius: 6px;
            cursor: pointer;
        }
        .btn:hover {
            background: #0056b3;
        }
        #msg {
            margin-top: 12px;
            text-align: center;
            font-weight: bold;
        }
        .err {
            color: crimson;
        }
        .ok {
            color: green;
        }
    </style>
</head>
```

```
<body>
<div class="box">
    <h2>Login Panel</h2>
    <input type="text" id="uname" class="input-field" placeholder="Username">
    <input type="password" id="pwd" class="input-field"
placeholder="Password">
    <button class="btn" onclick="loginCheck()">Login</button>
    <div id="msg"></div>
</div>

<script>
function loginCheck() {
    let uname = document.getElementById("uname").value.trim();
    let pwd = document.getElementById("pwd").value.trim();
    let msg = document.getElementById("msg");
    const validUser = "admin";
    const validPass = "1234";
    if (uname.length === 0 || pwd.length === 0) {
        msg.innerHTML = "Fields cannot be empty!";
        msg.className = "err";
        return;
    }
    if (uname.length < 4 || pwd.length < 4) {
        msg.innerHTML = "Enter at least 4 characters";
        msg.className = "err";
  return;
    }
    if (uname === validUser && pwd === validPass) {
        msg.innerHTML = "Welcome! Login successful";
```

```
        msg.className = "ok";
    } else {
        msg.innerHTML = "Wrong username or password";
        msg.className = "err";
    }
}
</script>
</body>
</html>
```







Result:

The "Data Retrieval & Sorting Dashboard" was successfully implemented. The application demonstrates efficient data handling using JavaScript array methods, providing a seamless user experience for managing records and viewing departmental statistics.

Task 4: Order Management System

Aim

To create and manage an Order Management System using SQL by implementing JOINs, Subqueries, and ORDER BY to identify:

- Customer order history
- Highest value order
- Most active customer

Algorithm

1. Create three tables: Customers, Products, and Orders.
2. Insert sample records into all tables.
3. Use JOIN to display customer order history.
4. Use a subquery to find the highest value order.
5. Use aggregation and subquery to find the most active customer.
6. Sort results using ORDER BY.

Program

Table Creation

```
CREATE TABLE Customers (

    customer_id INT PRIMARY KEY,

    name VARCHAR(50),

    city VARCHAR(50)

);



CREATE TABLE Products (

    product_id INT PRIMARY KEY,

    product_name VARCHAR(50),

    price DECIMAL(10,2)

);



CREATE TABLE Orders (

    order_id INT PRIMARY KEY,
```

```
    customer_id INT,
    product_id INT,
    quantity INT,
    order_date DATE,
    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id),
    FOREIGN KEY (product_id) REFERENCES Products(product_id)
);
INSERT INTO Customers VALUES
(1, 'Rahul', 'Bangalore'),
(2, 'Sneha', 'Hyderabad'),
(3, 'Karan', 'Pune');

INSERT INTO Products VALUES
(201, 'Tablet', 30000),
(202, 'Smartwatch', 8000),
(203, 'Bluetooth Speaker', 4000);

INSERT INTO Orders VALUES
(11, 1, 201, 2, '2024-04-05'),
(12, 2, 202, 3, '2024-04-10'),
(13, 3, 203, 4, '2024-04-12'),
(14, 1, 202, 1, '2024-04-15'),
(15, 2, 201, 1, ‘2024-04-18');
SELECT
    c.name AS Customer_Name,
    p.product_name,
    o.quantity,
```

```
    (p.price * o.quantity) AS Total_Amount,
    o.order_date
FROM Orders o
JOIN Customers c ON o.customer_id = c.customer_id
JOIN Products p ON o.product_id = p.product_id
ORDER BY o.order_date;
SELECT *
FROM (
    SELECT
        o.order_id,
        c.name,
        (p.price * o.quantity) AS Total_Amount
    FROM Orders o
    JOIN Customers c ON o.customer_id = c.customer_id
    JOIN Products p ON o.product_id = p.product_id
) AS OrderValues
WHERE Total_Amount = (
    SELECT MAX(p.price * o.quantity)
    FROM Orders o
    JOIN Products p ON o.product_id = p.product_id
);
SELECT name, order_count
FROM (
    SELECT
        c.name,
        COUNT(o.order_id) AS order_count
    FROM Customers c
```

```
    JOIN Orders o ON c.customer_id = o.customer_id
    GROUP BY c.name
) AS CustomerOrders
WHERE order_count = (
    SELECT MAX(order_count)
    FROM (
        SELECT COUNT(order_id) AS order_count
        FROM Orders
        GROUP BY customer_id
    ) AS OrderCounts
);
```

OUTPUT:

1. Customer Order History:

```
+---------------+------------------+----------+--------------+------------+
| Customer_Name | product_name     | quantity | Total_Amount | order_date |
+---------------+------------------+----------+--------------+------------+
| Rahul         | Tablet           |        2 |     60000.00 | 2024-04-05 |
| Sneha         | Smartwatch       |        3 |     24000.00 | 2024-04-10 |
| Karan         | Bluetooth Speaker |        4 |     16000.00 | 2024-04-12 |
| Rahul         | Smartwatch       |        1 |      8000.00 | 2024-04-15 |
| Sneha         | Tablet           |        1 |     30000.00 | 2024-04-18 |
+---------------+------------------+----------+--------------+------------+
5 rows in set (0.02 sec)
```

2. Highest Value Order:

```
+----------+-------+--------------+
| order_id | name  | Total_Amount |
+----------+-------+--------------+
|       11 | Rahul |     60000.00 |
+----------+-------+--------------+
1 row in set (0.03 sec)
```

3. Most Active Customer:

```
+-------+-------------+
| name  | order_count |
+-------+-------------+
| Rahul |           2 |
| Sneha |           2 |
+-------+-------------+
2 rows in set (0.02 sec)
```

Result

- Customer order history is displayed using JOIN.
- Highest value order is identified using subquery.
- Most active customer is determined using aggregation and subquery.

Task 5: Transaction-Based Payment Simulation

Aim

To simulate an online payment process using transactional management in Spring Boot, ensuring atomic operations where user balance is deducted and merchant balance is credited using COMMIT on success and ROLLBACK on failure.

Algorithm

- Create an **Account Entity** with fields: id, name, balance, type (USER/MERCHANT)
- Configure database and JPA properties
- Create a repository interface extending JpaRepository
- Create a service layer with a transactional method
- Deduct amount from user account
- Add amount to merchant account
- If both operations succeed → COMMIT
- If any failure occurs → ROLLBACK
- Create a runner class to simulate transaction execution

Program

Account Entity

```
import jakarta.persistence.*;

@Entity
public class Account {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private double balance;
```

```
    private String type; // USER or MERCHANT

    public Account() {}

    public Account(String name, double balance, String type) {
        this.name = name;
        this.balance = balance;
        this.type = type;
    }

    // Getters and Setters
    public Long getId() { return id; }
    public double getBalance() { return balance; }
    public void setBalance(double balance) { this.balance = balance; }
    public String getName() { return name; }
}
```

Repository Interface

```
import org.springframework.data.jpa.repository.JpaRepository;

public interface AccountRepository extends JpaRepository<Account,
Long> {
}
```

Service Layer (Transactional Logic)

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class PaymentService {

    @Autowired
    private AccountRepository repo;

    @Transactional
    public void transferMoney(Long userId, Long merchantId, double
amount) {

        Account user = repo.findById(userId)
```

```
            .orElseThrow(() -> new RuntimeException("User not
found"));

        Account merchant = repo.findById(merchantId)
            .orElseThrow(() -> new RuntimeException("Merchant not
found"));

        // Deduct from user
        if (user.getBalance() < amount) {
            throw new RuntimeException("Insufficient balance");
        }

        user.setBalance(user.getBalance() - amount);

        // Simulate failure (optional testing)
        // if (true) throw new RuntimeException("Transaction Failed");

        // Add to merchant
        merchant.setBalance(merchant.getBalance() + amount);

        repo.save(user);
        repo.save(merchant);
    }
}
```

Main Application (Runner)

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;

@Component
public class PaymentRunner implements CommandLineRunner {

    @Autowired
    private PaymentService service;

    @Autowired
    private AccountRepository repo;

    @Override
    public void run(String... args) throws Exception {
```

```
        // Create sample accounts
        Account user = new Account("Alice", 5000, "USER");
        Account merchant = new Account("Amazon", 10000,
"MERCHANT");

        repo.save(user);
        repo.save(merchant);
 try {
            service.transferMoney(user.getId(), merchant.getId(), 1000);
            System.out.println("Transaction Successful (COMMIT)");
        } catch (Exception e) {
            System.out.println("Transaction Failed (ROLLBACK)");
        }

        // Display updated balances
        repo.findAll().forEach(acc ->
            System.out.println(acc.getName() + " Balance: " +
acc.getBalance())
        );
    }
}
```

Output:

Result

The transaction-based payment simulation was successfully implemented using Spring Boot. The system ensures atomicity using transactional management, where either all operations are committed or none are applied, maintaining data consistency similar to real-world banking and digital payment systems.

Task 6: Automated Logging using Triggers & Views

Aim

To create a database trigger that automatically logs every INSERT or UPDATE operation performed on a table, and to create a view that summarizes daily activity logs for auditing purposes.

Algorithm

1. Create a main table (`employees`).
2. Create a log table (`employees_log`) to store action details.
3. Create a trigger for INSERT:
    - Capture operation type.
    - Store new data in log table.
4. Create a trigger for UPDATE:
    - Capture old and new values.
    - Store update details in log table.
5. Create a view (`daily_activity_report`) to summarize actions per day.
6. Perform INSERT and UPDATE operations.
7. Query logs and view to verify results.

Program

1. Create Database

```
CREATE DATABASE audit_lab;
USE audit_lab;
```

2. Create Main Table

```
CREATE TABLE employees (
    emp_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(50),
    department VARCHAR(50),
    salary DECIMAL(10,2),
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    ON UPDATE CURRENT_TIMESTAMP
);
```

3. Create Log Table

```
CREATE TABLE employees_log (
    log_id INT PRIMARY KEY AUTO_INCREMENT,
    emp_id INT,
    action_type VARCHAR(10),
    action_time DATETIME DEFAULT CURRENT_TIMESTAMP,
    old_salary DECIMAL(10,2),
    new_salary DECIMAL(10,2)
);
```

4. Trigger for INSERT

```
DELIMITER $$

CREATE TRIGGER trg_employees_insert
AFTER INSERT ON employees
FOR EACH ROW
BEGIN
    INSERT INTO employees_log(emp_id, action_type,
new_salary)
    VALUES (NEW.emp_id, 'INSERT', NEW.salary);
END $$

DELIMITER ;
```

5. Trigger for UPDATE

```
DELIMITER $$

CREATE TRIGGER trg_employees_update
AFTER UPDATE ON employees
FOR EACH ROW
BEGIN
    INSERT INTO employees_log(emp_id, action_type,
old_salary, new_salary)
    VALUES (NEW.emp_id, 'UPDATE', OLD.salary, NEW.salary);
END $$

DELIMITER ;
```

6. Create View

```
CREATE VIEW daily_activity_report AS
SELECT
    DATE(action_time) AS action_date,
    action_type,
    COUNT(*) AS total_actions
FROM employees_log
GROUP BY DATE(action_time), action_type
ORDER BY action_date;
```

7. Insert Records

```
INSERT INTO employees (name, department, salary)
VALUES
('Alice', 'HR', 40000),
('Bob', 'IT', 50000);
```

8. Update Record

```
UPDATE employees
SET salary = 52000
WHERE name = 'Bob';
```

9. Check Logs

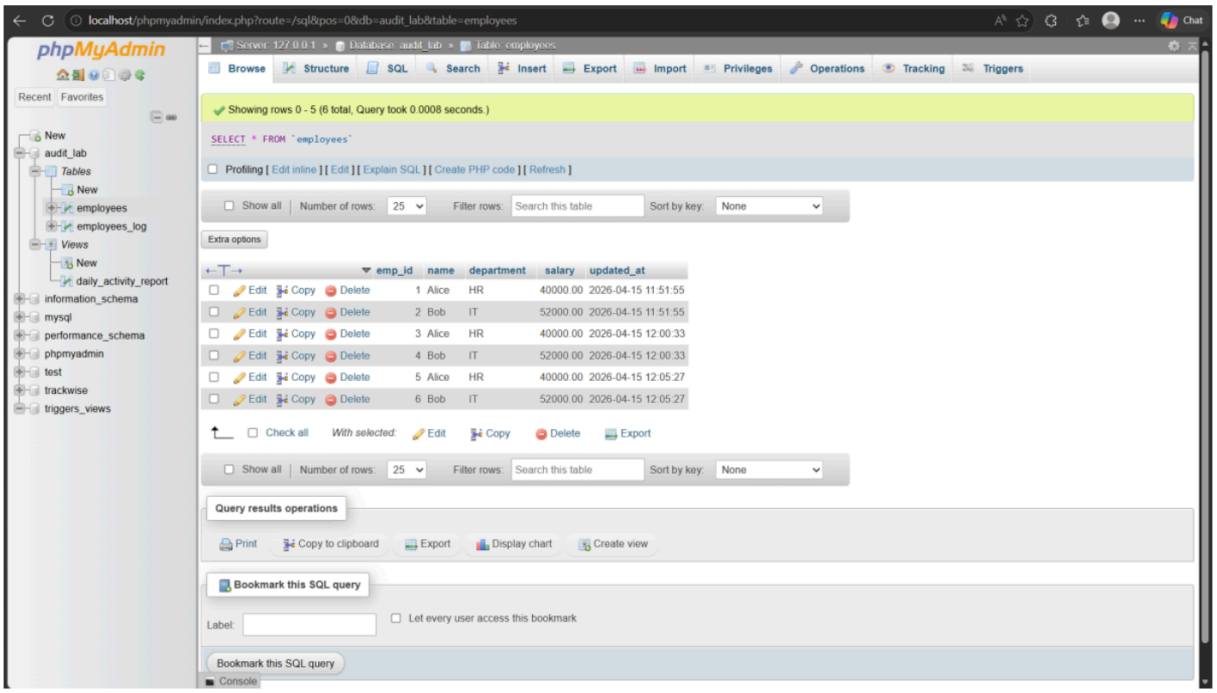
```
SELECT * FROM employees_log;
```

10. View Daily Report

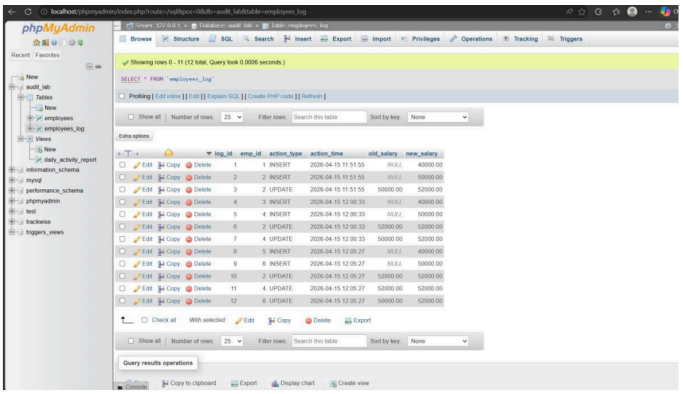
```
SELECT * FROM daily_activity_report;
```

<input type="email" id="email" placeholder="Email" onkeyup="validateEmail()">

****



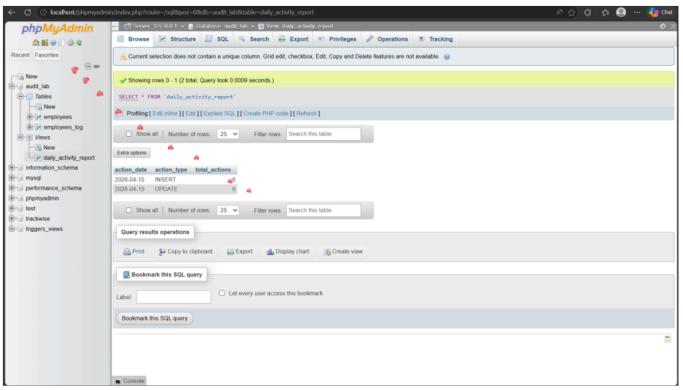
<textarea id="msg" placeholder="Feedback" onkeyup="validateMsg()"></textarea>



<span id="mErr" class="error">

<button type="button"

VIEWS



Result

- INSERT and UPDATE operations are automatically logged using triggers.
- Log table stores action type, timestamps, and salary changes.
- View provides a summarized daily activity report.

Task 7: Interactive Web Form with Events & Functions

AIM:

To design and develop an interactive web-based feedback form using HTML, CSS, and JavaScript that performs real-time input validation, enhances user interaction through event handling (keypress, mouse hover, and double-click), and ensures reusable validation using JavaScript functions.

ALGORITHM:

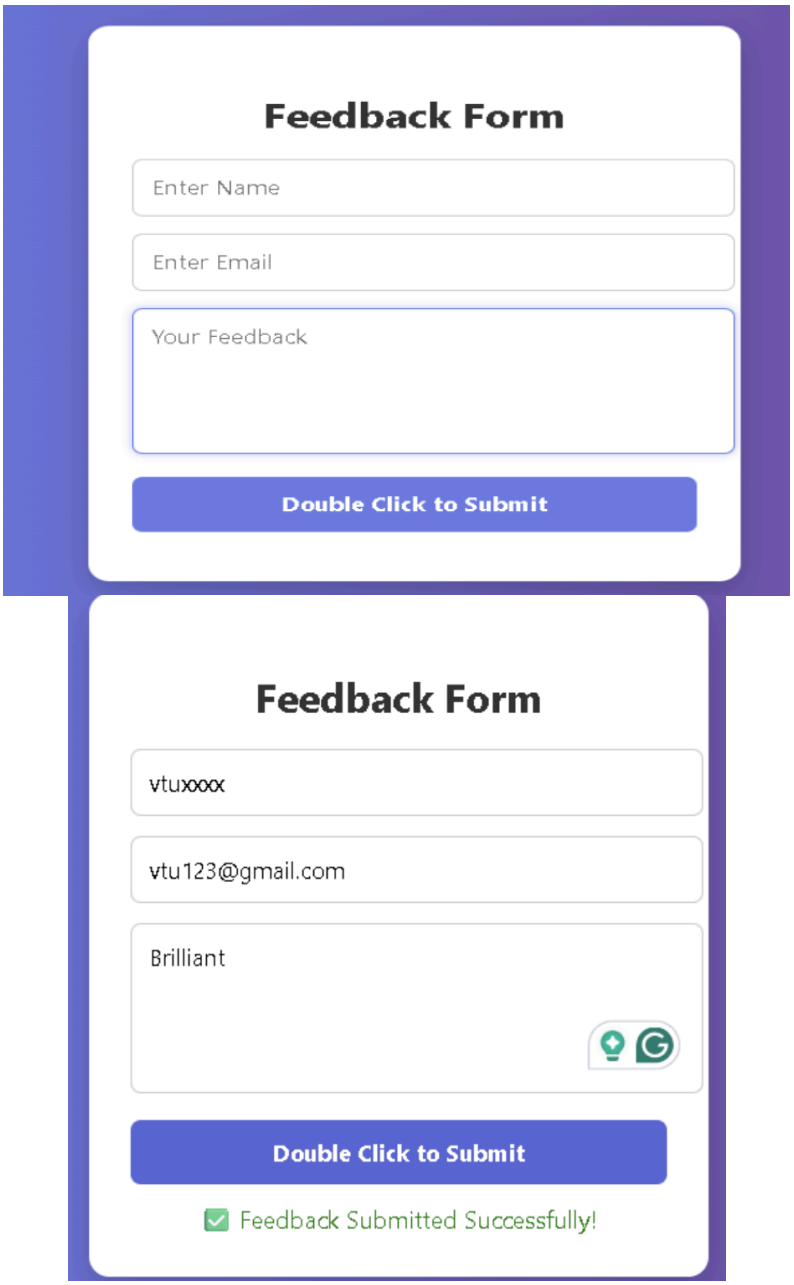
1. **Start the program**
2. **Create the HTML structure**
3. **Apply CSS styling**
4. **Implement keypress event validation**
    - On each key press (onkeyup):
        - Call validation functions
5. **Display validation messages**
6. **Create reusable JavaScript functions**
    - validateName()
    - validateEmail()
    - validateFeedback()
7. **Handle mouse hover event**
8. **Implement double-click submit**
    - Use ondblclick event on submit button
    - Call submitForm() function
9. **Validate all inputs before submission**
10. **Display success message**
11. **End the program**

PROGRAM:

```
<!DOCTYPE html>
<html>
<head>
    <title>Feedback Form</title>
    <style>
        input, textarea {
            display: block;
            margin: 10px;
            padding: 8px;
        }
        input:hover, textarea:hover {
            border: 2px solid blue; /* Hover effect */
        }
        .error {
            color: red;
            font-size: 12px;
        }
    </style>
</head>
<body>
<h3>Feedback Form</h3>
<form>
    <input type="text" id="name" placeholder="Name"
onkeyup="validateName()">
    <span id="nErr" class="error"></span>
```

```
ondblclick="submitForm()">Submit</button>
</form>
<script>
function validateName() {
    let n = document.getElementById("name").value;
    document.getElementById("nErr").innerHTML =
        (n.length < 3) ? "Min 3 chars" : "";
}
function validateEmail() {
    let e = document.getElementById("email").value;
    let pattern = /^[^ ]+@[^ ]+\.[a-z]{2,3}$/;
    document.getElementById("eErr").innerHTML =
        (!e.match(pattern)) ? "Invalid email" : "";
}
function validateMsg() {
    let m = document.getElementById("msg").value;
    document.getElementById("mErr").innerHTML = (m === "") ? "Required" : "";
}
function submitForm() {
    if (
        document.getElementById("nErr").innerHTML === "" &&
        document.getElementById("eErr").innerHTML === "" &&
        document.getElementById("mErr").innerHTML === ""
    ) {
        alert("Feedback Submitted!");
    } else {
        alert("Fix errors first!");
```

```
    }
}
</script>
</body>
</html>
```



RESULT:

The interactive feedback form was successfully designed and implemented using HTML, CSS, and JavaScript. The application performs real-time validation of user inputs using keypress events, ensuring that incorrect data is immediately identified and corrected.

Task 8: Employee Management Module using Spring Core

Aim

To create a simple Employee Management module using Spring Core that demonstrates Inversion of Control (IoC) and Dependency Injection (DI) using annotations such as `@Component` and `@Autowired`. The application uses `ApplicationContext` (BeanFactory) to manage beans and stores employee data in memory using a `HashMap`.

Algorithm

1. Start
2. Create a Maven project with Spring Boot web dependency
3. Define an Employee model class with fields: id, name, department, salary
4. Create EmployeeDAO interface with CRUD methods
5. Implement DAO using HashMap and annotate with `@Component`
6. Create EmployeeService and inject DAO using `@Autowired`
7. Create EmployeeController and expose REST APIs
8. Use Spring Boot main class to enable IoC
9. Create index.html UI
10. Run application on localhost:8080
11. Perform CRUD operations
12. Display results
13. Stop

Procedure

1. Create Maven project (Group: `com.employee`, Artifact: `EmployeeManagement`)
2. Add Spring Boot dependency in `pom.xml`
3. Create model class `Employee`
4. Create DAO interface
5. Implement DAO using HashMap
6. Create service layer with DI
7. Create controller with REST APIs
8. Configure main application class
9. Create frontend UI
10. Run and test application

Program

1. pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
```

```
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.1.5</version>
    </parent>

    <groupId>com.employee</groupId>
    <artifactId>EmployeeManagement</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <java.version>17</java.version>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</
artifactId>
        </dependency>
    </dependencies>
</project>
```

2. Employee.java

```
package com.employee.model;

public class Employee {
    private int id;
    private String name;
    private String department;
    private double salary;

    public Employee() {}

    public Employee(int id, String name, String department,
double salary) {
        this.id = id;
        this.name = name;
        this.department = department;
        this.salary = salary;
    }
```

```
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getDepartment() { return department; }
    public void setDepartment(String department)
{ this.department = department; }

    public double getSalary() { return salary; }
    public void setSalary(double salary) { this.salary =
salary; }
}
```

3. EmployeeDAO.java

```
package com.employee.dao;

import com.employee.model.Employee;
import java.util.List;

public interface EmployeeDAO {
    void addEmployee(Employee employee);
    Employee getEmployeeById(int id);
    List<Employee> getAllEmployees();
    void updateEmployee(Employee employee);
    void deleteEmployee(int id);
}
```

4. EmployeeDAOImpl.java

```
package com.employee.dao;

import com.employee.model.Employee;
import org.springframework.stereotype.Component;
import java.util.*;

@Component
public class EmployeeDAOImpl implements EmployeeDAO {

    private Map<Integer, Employee> employeeMap = new
HashMap<>();
```

```
    public void addEmployee(Employee employee) {
        employeeMap.put(employee.getId(), employee);
    }

    public Employee getEmployeeById(int id) {
        return employeeMap.get(id);
    }

    public List<Employee> getAllEmployees() {
        return new ArrayList<>(employeeMap.values());
    }

    public void updateEmployee(Employee employee) {
        employeeMap.put(employee.getId(), employee);
    }

    public void deleteEmployee(int id) {
        employeeMap.remove(id);
    }
}
```

5. EmployeeService.java

```
package com.employee.service;

import com.employee.dao.EmployeeDAO;
import com.employee.model.Employee;
import
org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;
import java.util.List;

@Component
public class EmployeeService {

    @Autowired
    private EmployeeDAO employeeDAO;

    public void addEmployee(Employee employee) {
        employeeDAO.addEmployee(employee);
    }

    public Employee getEmployeeById(int id) {
        return employeeDAO.getEmployeeById(id);
```

```
    }

    public List<Employee> getAllEmployees() {
        return employeeDAO.getAllEmployees();
    }

    public void updateEmployee(Employee employee) {
        employeeDAO.updateEmployee(employee);
    }

    public void deleteEmployee(int id) {
        employeeDAO.deleteEmployee(id);
    }
}
```

6. EmployeeController.java

```
package com.employee.controller;

import com.employee.model.Employee;
import com.employee.service.EmployeeService;
import
org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/employees")
@CrossOrigin("*")
public class EmployeeController {

    @Autowired
    private EmployeeService service;

    @GetMapping
    public List<Employee> getAll() {
        return service.getAllEmployees();
    }

    @GetMapping("/{id}")
    public Employee getById(@PathVariable int id) {
        return service.getEmployeeById(id);
    }

    @PostMapping
```

```
    public String add(@RequestBody Employee e) {
        service.addEmployee(e);
        return "Employee Added";
    }

    @PutMapping("/{id}")
    public String update(@PathVariable int id, @RequestBody
Employee e) {
        e.setId(id);
        service.updateEmployee(e);
        return "Employee Updated";
    }

    @DeleteMapping("/{id}")
    public String delete(@PathVariable int id) {
        service.deleteEmployee(id);
        return "Employee Deleted";
    }
}
```

7. MainApp.java

```
package com.employee;

import org.springframework.boot.SpringApplication;
import
org.springframework.boot.autoconfigure.SpringBootApplicatio
n;

@SpringBootApplication
public class MainApp {
    public static void main(String[] args) {
        SpringApplication.run(MainApp.class, args);
    }
}
```

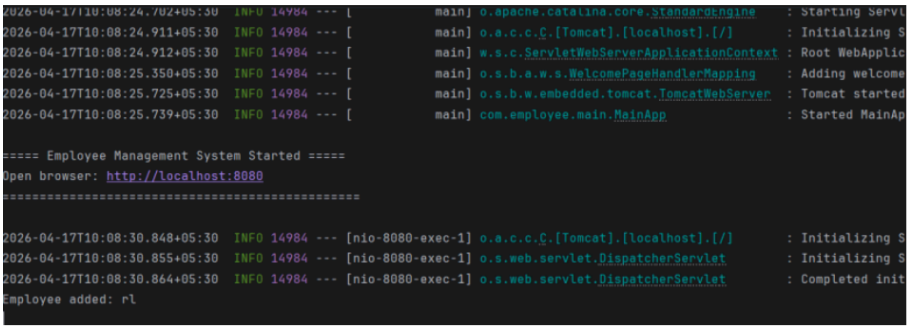
8. application.properties

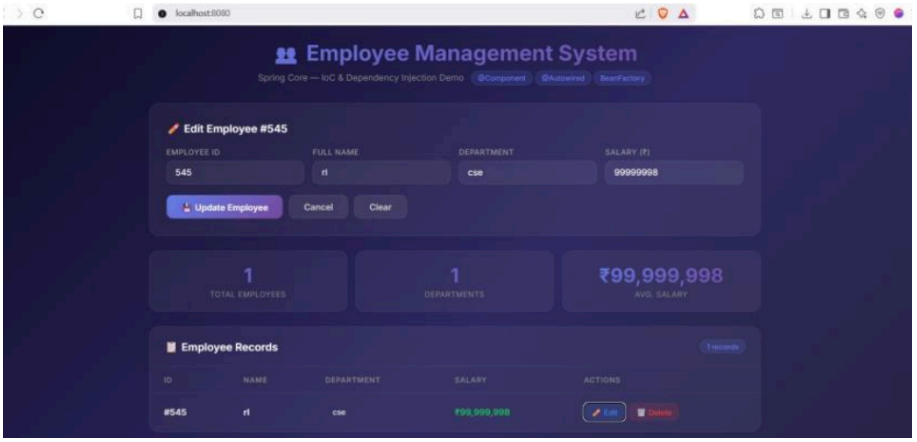
```
server.port=8080
spring.application.name=EmployeeManagement
```

9. index.html (UI)

```
<!DOCTYPE html>
<html>
<head>
    <title>Employee Management</title>
</head>
<body>
<h2>Employee Management</h2>

<input id="id" placeholder="ID"><br>
<input id="name" placeholder="Name"><br>
<input id="dept" placeholder="Department"><br>
<input id="salary" placeholder="Salary"><br>

<button onclick="add()">Add</button>

<script>
function add(){
 fetch('/api/employees',{
  method:'POST',
  headers:{'Content-Type':'application/json'},
  body: JSON.stringify({
   id:parseInt(id.value),
   name:name.value,
   department:dept.value,
   salary:parseFloat(salary.value)
  })
 }).then(()=>alert("Added"));
}
</script>

</body>
</html>
```

OUTPUT:





Result

- Employee CRUD operations are successfully performed
- IoC achieved via Spring container
- DI achieved using `@Autowired`
- Data stored in-memory using HashMap
- REST API + UI working on localhost

Task 9: Basic Spring MVC Application with Annotation-Based Configuration

Aim

To develop a basic Spring MVC application using annotation-based configuration without XML files, create a controller to accept user requests, and display employee details following the Model-View-Controller (MVC) architecture.

Procedure

1. Set up a Maven project and add required dependencies.
2. Configure Spring using annotations instead of XML.
3. Create a model class (Employee/Person).
4. Create a controller using `@Controller`.
5. Handle requests using `@GetMapping`.
6. Pass data from controller to view using `Model`.
7. Configure view resolver (Thymeleaf/JSP).
8. Create view pages to display data.
9. Run application and verify output in browser.

Program

1. pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.2.1</version>
    </parent>

    <groupId>com.example</groupId>
    <artifactId>SpringMVCApp</artifactId>
    <version>0.0.1-SNAPSHOT</version>

    <properties>
        <java.version>21</java.version>
    </properties>

    <dependencies>
        <!-- Spring MVC -->
```

```
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</
artifactId>
        </dependency>

        <!-- View Engine -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-thymeleaf</
artifactId>
        </dependency>
    </dependencies>
</project>
```

2. Main Application

```
package edu.jfs.app;

import org.springframework.boot.SpringApplication;
import
org.springframework.boot.autoconfigure.SpringBootApplicatio
n;

@SpringBootApplication
public class SpringMVCSampleApplication {
    public static void main(String[] args) {

SpringApplication.run(SpringMVCSampleApplication.class,
args);
    }
}
```

3. Model Class (Person.java)

```
package edu.jfs.app;

public class Person {
    private int id;
    private String nameString;

    public Person() {}
```

```
    public Person(int id, String nameString) {
        this.id = id;
        this.nameString = nameString;
    }

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getNameString() { return nameString; }
    public void setNameString(String nameString) {
        this.nameString = nameString;
    }
}
```

4. Controller (MVCController.java)

```
package edu.jfs.app;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class MVCController {

    @GetMapping("/")
    public String home(Model model) {
        model.addAttribute("str", "USER1");
        return "OneString";
    }

    @GetMapping("/person")
    public String person(Model model) {
        Person p = new Person(101, "USER1");
        model.addAttribute("str", p.getNameString());
        return "OneString";
    }

    @GetMapping("/show")
    public String show(Model model) {
        Person p = new Person(101, "USER1");
        model.addAttribute("person", p);
        return "Person";
    }
}
```

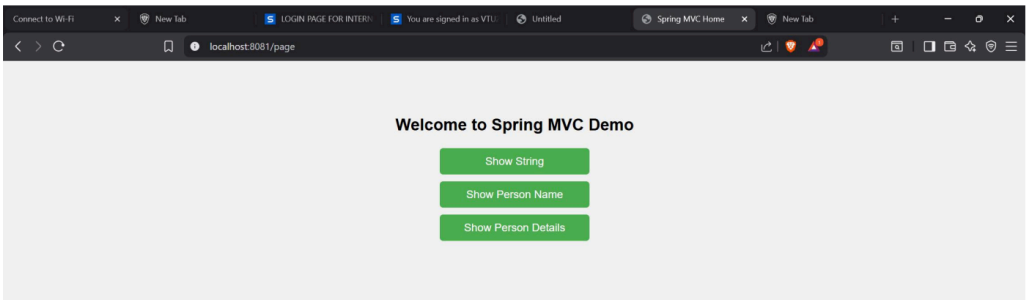
5. View (OneString.html)

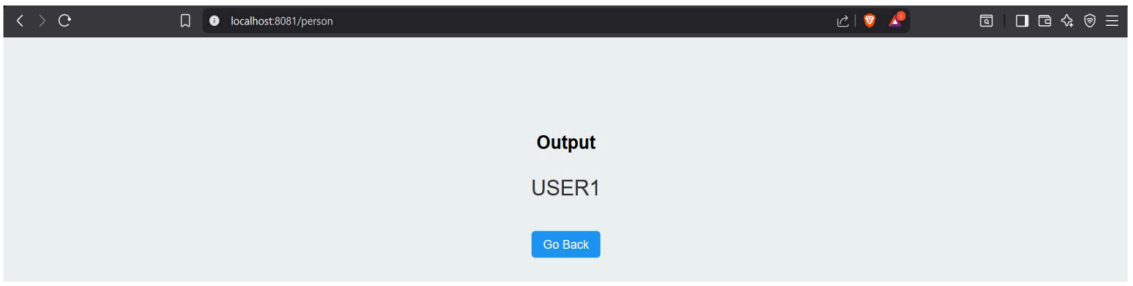
```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<body>
    <h2 th:text="${str}"></h2>
</body>
</html>
```

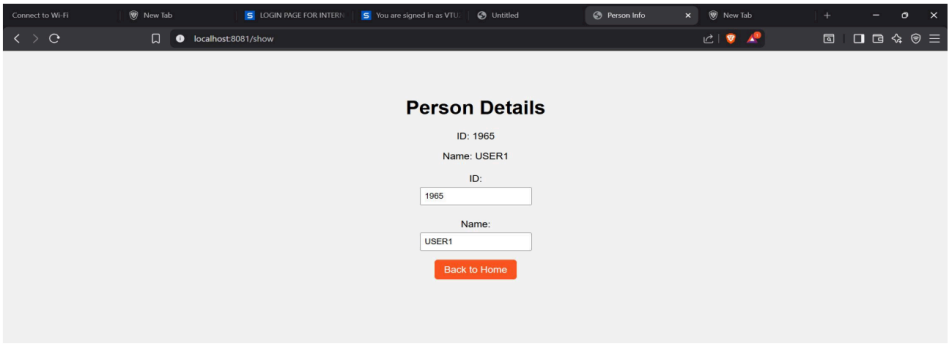
6. View (Person.html)

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<body>
    <h2>Person Details</h2>
    <p>ID: <span th:text="${person.id}"></span></p>
    <p>Name: <span th:text="${person.nameString}"></span></
p>
</body>
</html>
```

OUTPUT:







Result

- Spring MVC application runs successfully
- Controller handles requests using annotations
- Data is passed using Model
- View displays dynamic content using Thymeleaf
- MVC flow is achieved

Task 10:Spring MVC Application Using Annotation-Based Configuration

Aim

To develop a Spring MVC application using annotation-based configuration (without XML) that handles user requests through a controller and displays data using the MVC architecture.

Objective

- Understand Spring MVC architecture
- Use annotations instead of XML configuration
- Implement Controller, Model, and View
- Handle HTTP requests using `@GetMapping`

Algorithm

1. Start
2. Create a Spring Boot Maven project
3. Add Spring Web dependency
4. Create model class
5. Create controller using `@Controller`
6. Map URLs using `@GetMapping`
7. Pass data using `Model`
8. Create view using Thymeleaf
9. Run application
10. Verify output in browser
11. Stop

Program

1. pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.2.1</version>
    </parent>

    <groupId>com.example</groupId>
```

```
    <artifactId>SpringMVCApp</artifactId>
    <version>0.0.1-SNAPSHOT</version>

    <properties>
        <java.version>21</java.version>
    </properties>

    <dependencies>
        <!-- Spring MVC -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</
artifactId>
        </dependency>

        <!-- Thymeleaf -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-thymeleaf</
artifactId>
        </dependency>
    </dependencies>
</project>
```

2. Main Class

```
package com.example;

import org.springframework.boot.SpringApplication;
import
org.springframework.boot.autoconfigure.SpringBootApplicatio
n;

@SpringBootApplication
public class SpringMVCApplication {
    public static void main(String[] args) {
        SpringApplication.run(SpringMVCApplication.class,
args);
    }
}
```

3. Model Class

```
package com.example.model;

public class Employee {
    private int id;
    private String name;
    private String department;

    public Employee(int id, String name, String department)
{
        this.id = id;
        this.name = name;
        this.department = department;
    }

    public int getId() { return id; }
    public String getName() { return name; }
    public String getDepartment() { return department; }
}
```

4. Controller

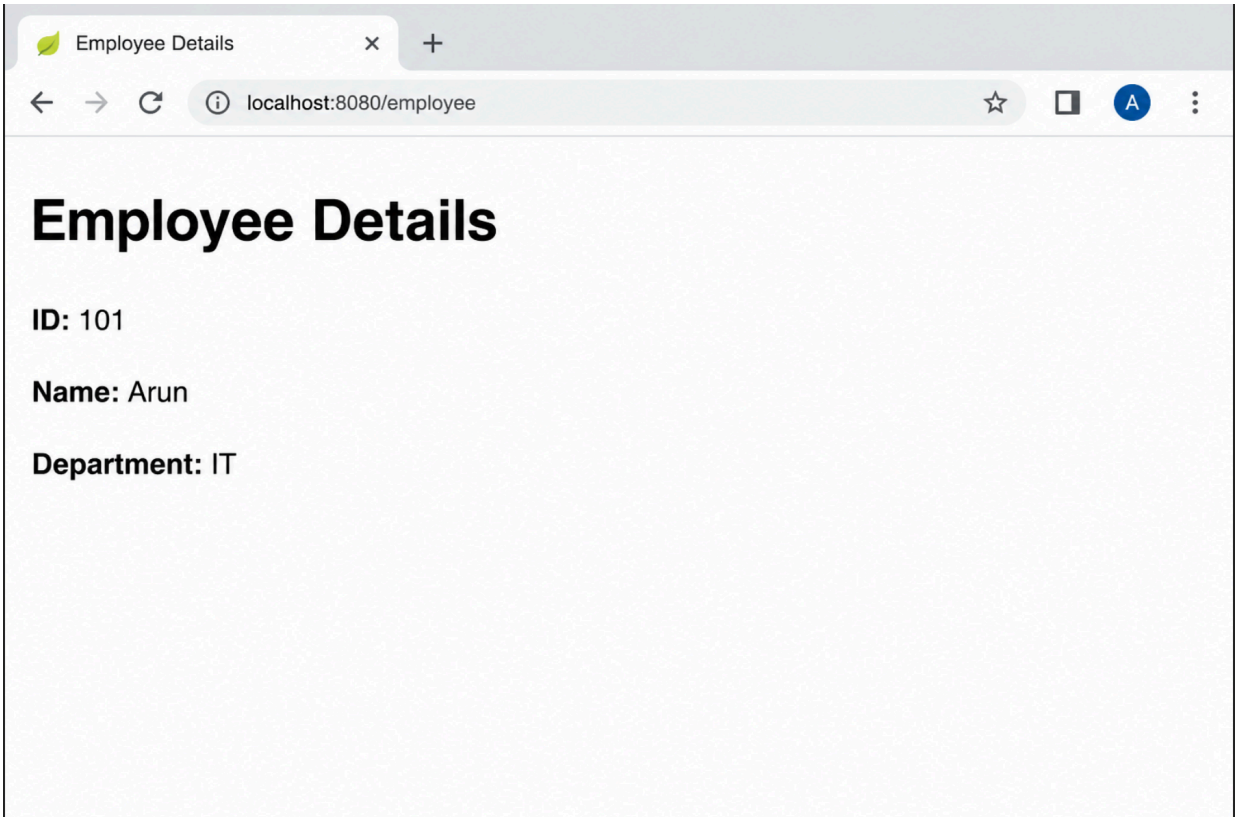
```
package com.example.controller;

import com.example.model.Employee;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class EmployeeController {

    @GetMapping("/employee")
    public String getEmployee(Model model) {
        Employee emp = new Employee(101, "Arun", "IT");
        model.addAttribute("employee", emp);
        return "employee";
    }
}
```

5. View (employee.html)

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
```

```
<body>
    <h2>Employee Details</h2>
    <p>ID: <span th:text="${employee.id}"></span></p>
    <p>Name: <span th:text="${employee.name}"></span></p>
    <p>Department: <span th:text="$
{employee.department}"></span></p>
</body>
</html>
```

OUTPUT:



Result:

- Spring MVC application created successfully
- Controller handles request /employee
- Model data passed to view
- View displays employee details

Task 11:

Implement a Data Access Layer using Spring Data JPA. Use JpaRepository and custom query methods to retrieve student records based on conditions such as department or age, and include sorting and pagination

Aim

To design and implement a Data Access Layer using Spring Data JPA, leveraging JpaRepository and custom query methods to retrieve student records based on conditions (department, age) with sorting and pagination.

Algorithm

1. Create a Student Entity with fields: id, name, age, department
2. Configure JPA properties in application file
3. Create an interface extending JpaRepository
4. Define custom query methods:
    - Find by department
    - Find by age greater than a value
5. Use sorting via Sort class
6. Use pagination via Pageable
7. Create a service layer to call repository methods
8. Run application and fetch filtered results

Program

1. **Student Entity**

```
import jakarta.persistence.*;

@Entity
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
```

```
    private int age;
    private String department;

    // Constructors
    public Student() {}

    public Student(String name, int age, String department) {
        this.name = name;
        this.age = age;
        this.department = department;
    }

    // Getters & Setters
}
```

2. Repository Interface

```
import org.springframework.data.jpa.repository.JpaRepository;

import org.springframework.data.domain.*;


import java.util.List;

public interface StudentRepository extends JpaRepository<Student, Long> {

    // Custom query methods

    List<Student> findByDepartment(String department);


    List<Student> findByAgeGreaterThan(int age);
```

1. Service Layer

```
import org.springframework.beans.factory.annotation.Autowired;

import org.springframework.data.domain.*;

import org.springframework.stereotype.Service;


import java.util.List;

@Service

public class StudentService {
```

```
    @Autowired
    private StudentRepository repo;

    public List<Student> getByDepartment(String dept) {
        return repo.findByDepartment(dept);
    }

    public List<Student> getByAge(int age) {
        return repo.findByAgeGreaterThan(age);
    }

    public Page<Student> getPaginated(String dept, int page, int size) {
        Pageable pageable = PageRequest.of(page, size,
Sort.by("age").ascending());
        return repo.findByDepartment(dept, pageable);
    }
}
```

2. Main Application (Runner)

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;

@Component
public class TestRunner implements CommandLineRunner {

    @Autowired
    private StudentService service;

    @Override
 public void run(String... args) throws Exception {
```

```
        System.out.println("Students in CSE:");
        service.getByDepartment("CSE").forEach(s ->
            System.out.println(s.getName() + " " + s.getAge())
        );

        System.out.println("\nStudents age > 20:");
        service.getByAge(20).forEach(s ->
            System.out.println(s.getName())
        );

        System.out.println("\nPaginated Result:");
        service.getPaginated("CSE", 0, 2).forEach(s ->
            System.out.println(s.getName())
        );
    }
}
```

OUTPUT:

1. Get Students by Department (CSE)


```
{
    "status: 200,
    "message": "Students fetched successfully",
    "data": {
     {
        "id": 1,
        "name": "Arun",
        "age": 21,
        "department": "CSE"
      },
     {
        "id": 2,
        "name": "Priya",
        "age": 22,
        "department": "CSE"
      }
}
```


2. Get Students with Age > 20


```
{
    "status: 200,
    "message": "Students filtered by age",
    "data": {
     {
        "id": 1,
        "name": "Arun",
        "age": 21,
        "department": "CSE"
      },
     {
        "id": 2,
        "name": "Priya",
        "age": 22,
        "department": "CSE"
      }
}
```


Result:The Data Access Layer was successfully implemented using Spring Data JPA, enabling efficient retrieval of student records using custom query methods, along with sorting and pagination for optimized data handling.

Task 12: RESTful API for Product Management using Spring Boot

Aim

To design and develop a RESTful API for Product Management using Spring Boot that performs CRUD operations (GET, POST, PUT, DELETE) and returns JSON responses. The API is tested using a REST client like Postman.

Algorithm

1. Create Spring Boot project with required dependencies
2. Configure database connection
3. Create Product entity class
4. Create repository using JPA
5. Create service class
6. Create REST controller
7. Implement CRUD operations
8. Run application
9. Test APIs using Postman

Program

1. Product Entity

```
package com.example.product;

import jakarta.persistence.*;

@Entity
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private double price;
    private int quantity;

    public Product() {}

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getName() { return name; }
```

```
    public void setName(String name) { this.name = name; }

    public double getPrice() { return price; }
    public void setPrice(double price) { this.price = price; }

    public int getQuantity() { return quantity; }
    public void setQuantity(int quantity) { this.quantity = quantity; }
}
```

2. Repository

```
package com.example.product;

import org.springframework.data.jpa.repository.JpaRepository;

public interface ProductRepository extends JpaRepository<Product, Long> {
}
```

3. Service

```
package com.example.product;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class ProductService {

    @Autowired
    private ProductRepository repo;

    public List<Product> getAllProducts() {
        return repo.findAll();
    }

    public Product getProductById(Long id) {
        return repo.findById(id).orElse(null);
    }

    public Product saveProduct(Product product) {
        return repo.save(product);
    }

    public Product updateProduct(Long id, Product product) {
        Product p = repo.findById(id).orElse(null);
```

```
        if (p != null) {
            p.setName(product.getName());
            p.setPrice(product.getPrice());
            p.setQuantity(product.getQuantity());
            return repo.save(p);
        }
        return null;
    }

    public void deleteProduct(Long id) {
        repo.deleteById(id);
    }
}
```

4. Controller (Corrected)

```
package com.example.product;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/products")
public class ProductController {

    @Autowired
    private ProductService service;

    @GetMapping
    public List<Product> getAllProducts() {
        return service.getAllProducts();
    }

    @GetMapping("/{id}")
    public Product getProduct(@PathVariable Long id) {
        return service.getProductById(id);
    }

    @PostMapping
    public Product createProduct(@RequestBody Product product) {
        return service.saveProduct(product);
    }

    @PutMapping("/{id}")
    public Product updateProduct(@PathVariable Long id, @RequestBody Product product) {
        return service.updateProduct(id, product);
    }
```

```
    @DeleteMapping("/{id}")
    public String deleteProduct(@PathVariable Long id) {
        service.deleteProduct(id);
        return "Deleted Successfully";
    }
}
```

OUTPUT:


```
{
  "id": 3,
  "name": "Headphones",
  "price": 1500.0,
  "quantity": 20
}
```

```
{
  "id": 1,
  "name": "Updated Laptop",
  "price": 55000.0,
  "quantity": 4
}
```


RESULT: The RESTful API for Product Management was successfully developed using Spring Boot. CRUD operations were implemented and tested using a REST client, confirming proper functionality and JSON-based communication.

Task: 13 Exception Handling & Validation in REST API

Aim

To implement global exception handling and apply input validation in a RESTful application to ensure reliable data processing and meaningful error responses using Spring Boot.

Procedure

1. Create a Spring Boot project using Spring Initializr
2. Add dependencies:
    - Spring Web
    - Spring Boot Validation
    - Lombok (optional)
3. Create a model class with validation annotations
4. Create REST controller using `@Valid`
5. Implement global exception handling using `@RestControllerAdvice`
6. Define custom exception class
7. Return structured error responses
8. Test API using Postman or curl

Program

1. Model Class (User.java)

```
package com.example.validation;

import jakarta.validation.constraints.*;

public class User {

    @NotNull(message = "ID cannot be null")
    private Long id;

    @NotBlank(message = "Name is required")
    @Size(min = 3, message = "Name must be at least 3
characters")
    private String name;

    @Email(message = "Invalid email format")
    private String email;
```

```
    @Min(value = 18, message = "Age must be at least 18")
    private int age;

    // Getters and Setters
}
```

2. REST Controller

```
package com.example.validation;

import org.springframework.web.bind.annotation.*;
import jakarta.validation.Valid;

@RestController
@RequestMapping("/users")
public class UserController {

    @PostMapping
    public String createUser(@Valid @RequestBody User user)
{
        return "User created successfully";
    }

    @GetMapping("/{id}")
    public String getUser(@PathVariable Long id) {
        if (id <= 0) {
            throw new ResourceNotFoundException("Invalid
User ID");
        }
        return "User found";
    }
}
```

3. Custom Exception

```
package com.example.validation;

public class ResourceNotFoundException extends
RuntimeException {
    public ResourceNotFoundException(String message) {
        super(message);
    }
}
```

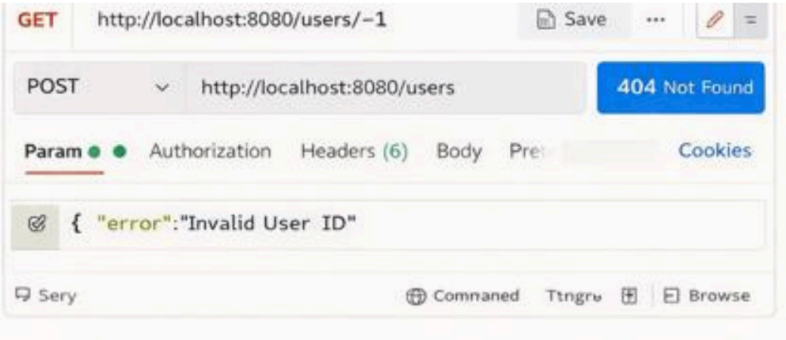
4. Global Exception Handler

```
package com.example.validation;

import org.springframework.http.HttpStatus;
import
org.springframework.web.bind.MethodArgumentNotValidExceptio
n;
import org.springframework.web.bind.annotation.*;

import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    // Validation errors

@ExceptionHandler(MethodArgumentNotValidException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public Map<String, String> handleValidationExceptions(
            MethodArgumentNotValidException ex) {

        Map<String, String> errors = new HashMap<>();


ex.getBindingResult().getFieldErrors().forEach(error ->
                errors.put(error.getField(),
error.getDefaultMessage())
        );

        return errors;
    }

    // Custom exception
    @ExceptionHandler(ResourceNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public Map<String, String> handleResourceNotFound(
            ResourceNotFoundException ex) {

        Map<String, String> error = new HashMap<>();
        error.put("error", ex.getMessage());
        return error;
    }
```

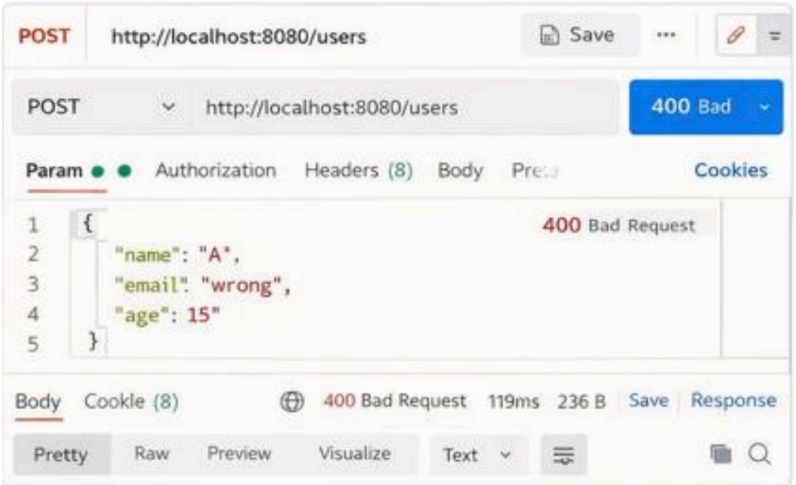
```
    // General exception
    @ExceptionHandler(Exception.class)
    @ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR)
    public Map<String, String>
handleGlobalException(Exception ex) {

        Map<String, String> error = new HashMap<>();
        error.put("error", "Something went wrong");
        return error;
    }
}
```

OUTPUT:

Custom exception output:



Validation error output:



Result : The RESTful application successfully:

- Validates input data using annotation-based validation
- • Handles exceptions globally using @RestControllerAdvice
- • Returns structured and meaningful error responses
- • Prevents invalid data from being processed Thus, the system becomes more robust, user-friendly, and production-ready.

Task 14: Microservices System with Two Independent Services (Spring Boot)

Aim:

To implement two independent microservices using Java Spring Boot that communicate using REST API.

Introduction:

Microservices architecture splits application into small independent services. Each service runs separately and communicates using HTTP REST APIs.

Algorithm:

1. Create User Service Spring Boot project.
2. Create Order Service Spring Boot project.
3. Run User Service on port 8081.
4. Run Order Service on port 8082.
5. Order Service calls User Service API.
6. Test using Postman.
7. Verify JSON output.

Program 1: User Service (Port 8081)

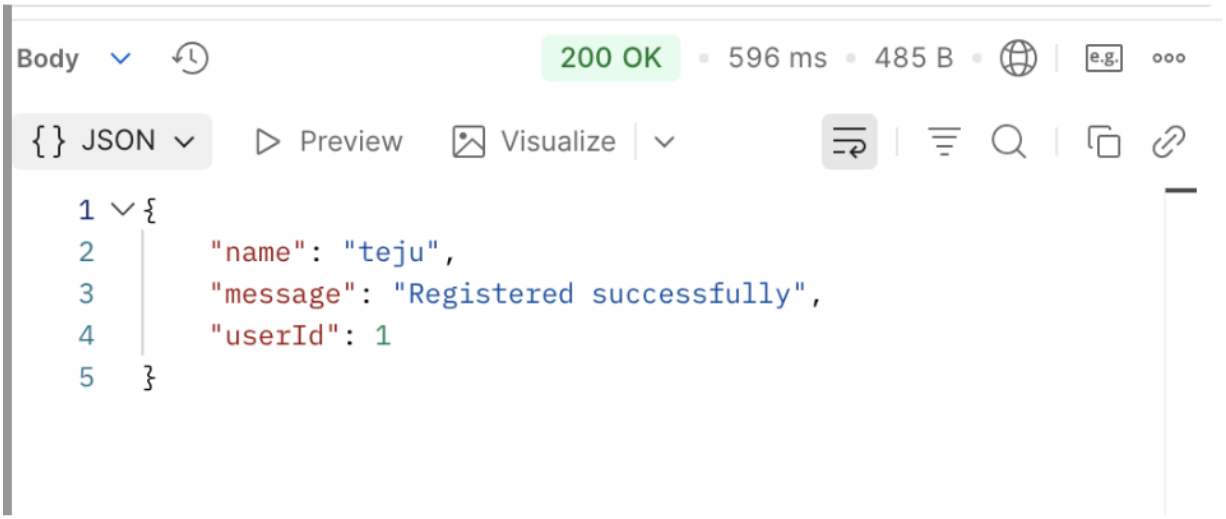
```
@RestController
@RequestMapping("/user")
public class UserController {

@PostMapping("/register")
public Map<String,Object> registerUser(@RequestBody Map<String,String>
data){

   Map<String,Object> res = new HashMap<>();
   res.put("name", data.get("name"));
   res.put("message","Registered successfully");
   res.put("userId",1);

   return res;
}
}
```

Program 2: Order Service (Port 8082)

```
@RestController
@RequestMapping("/order")
public class OrderController {
```

```
@GetMapping("/{id}")
public Map<String,Object> getOrder(@PathVariable int id){

    RestTemplate rt = new RestTemplate();

    String url = "http://localhost:8081/user/" + id;

    Map user = rt.getForObject(url, Map.class);

  Map<String,Object> order = new HashMap<>();
    order.put("orderId", id);
    order.put("userDetails", user);

    return order;
}
}
```

OUTPUT:



Result: Two independent microservices were created successfully using Spring Boot. Both services communicated using REST API and produced correct JSON output.

Task 15: Service Registry and Discovery using Eureka

Aim

To implement Service Registry and Service Discovery using Eureka by registering multiple microservices and enabling dynamic service lookup instead of using hard-coded service URLs.

Procedure

1. Create a Eureka Server (Service Registry)
2. Add Eureka Server dependencies
3. Enable Eureka Server using annotation
4. Create multiple microservices (clients)
5. Add Eureka Client dependency
6. Configure application properties
7. Enable service registration
8. Start Eureka Server
9. Run microservices
10. Verify services in Eureka Dashboard
11. Use service names for communication

Algorithm

1. Start
2. Create Eureka Server
3. Add `@EnableEurekaServer`
4. Run server on port 8761
5. Create microservices
6. Add Eureka Client dependency
7. Configure properties
8. Enable `@EnableDiscoveryClient`
9. Run services
10. Register with Eureka
11. Perform service discovery
12. Stop

Program

1. Eureka Server Application

```
package com.example.eureka;

import org.springframework.boot.SpringApplication;
```

```
import
org.springframework.boot.autoconfigure.SpringBootApplicatio
n;
import
org.springframework.cloud.netflix.eureka.server.EnableEurek
aServer;

@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApplication {
    public static void main(String[] args) {

SpringApplication.run(EurekaServerApplication.class, args);
    }
}
```

application.properties (Eureka Server)

```
server.port=8761

eureka.client.register-with-eureka=false
eureka.client.fetch-registry=false
```

2. Microservice 1 (Client)

```
package com.example.service1;

import org.springframework.boot.SpringApplication;
import
org.springframework.boot.autoconfigure.SpringBootApplicatio
n;
import
org.springframework.cloud.client.discovery.EnableDiscoveryC
lient;

@SpringBootApplication
@EnableDiscoveryClient
public class Service1Application {
    public static void main(String[] args) {
        SpringApplication.run(Service1Application.class,
args);
    }
}
```

application.properties (Service 1)

```
spring.application.name=service-1
server.port=8081

eureka.client.service-url.defaultZone=http://
localhost:8761/eureka/
```

3. Microservice 2 (Client)

```
package com.example.service2;

import org.springframework.boot.SpringApplication;
import
org.springframework.boot.autoconfigure.SpringBootApplicatio
n;
import
org.springframework.cloud.client.discovery.EnableDiscoveryC
lient;

@SpringBootApplication
@EnableDiscoveryClient
public class Service2Application {
    public static void main(String[] args) {
        SpringApplication.run(Service2Application.class,
args);
    }
}
```

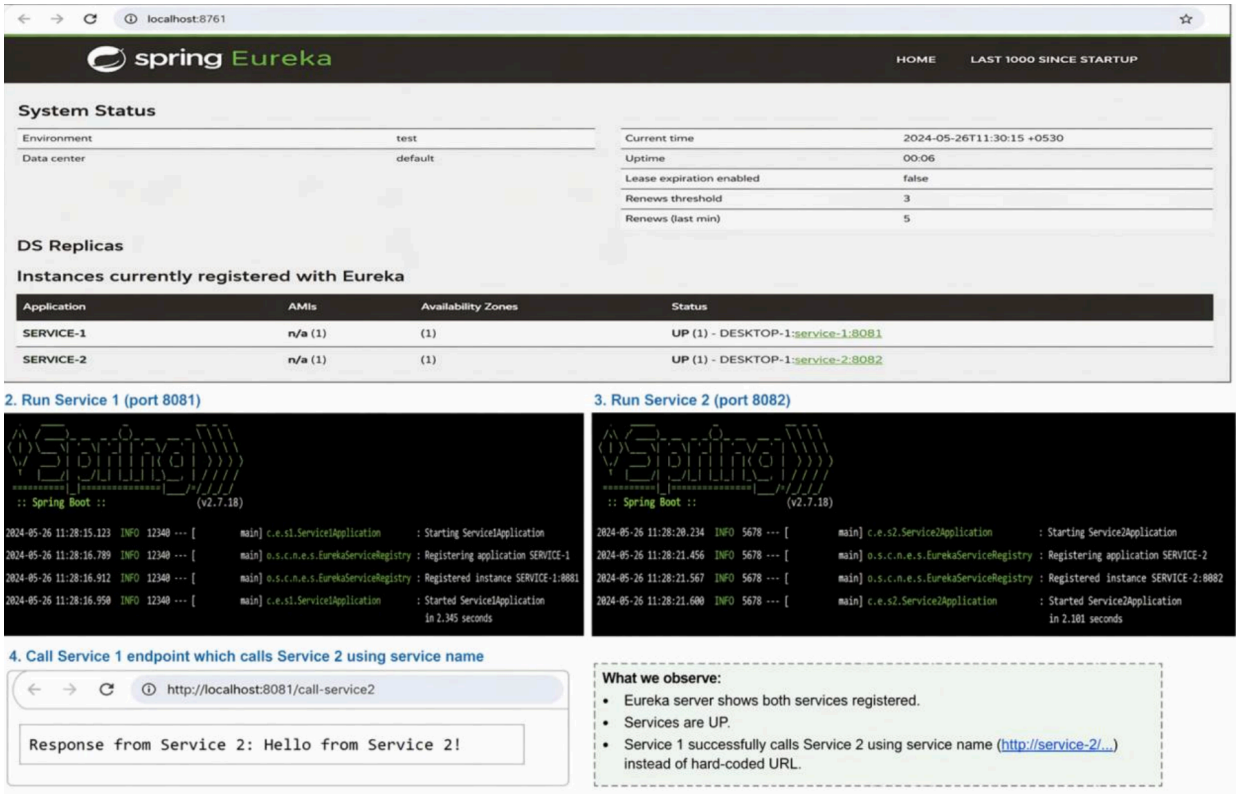
application.properties (Service 2)

```
spring.application.name=service-2
server.port=8082

eureka.client.service-url.defaultZone=http://
localhost:8761/eureka/
```

4. Service Communication using RestTemplate

```
package com.example.service1;

import org.springframework.context.annotation.Bean;
import
org.springframework.context.annotation.Configuration;
import
org.springframework.cloud.client.loadbalancer.LoadBalanced;
import org.springframework.web.client.RestTemplate;
import
org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

@Configuration
class AppConfig {
    @Bean
    @LoadBalanced
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@Service
class ServiceCaller {

    @Autowired
    private RestTemplate restTemplate;

    public String callService() {
        return restTemplate.getForObject("http://service-2/
endpoint", String.class);
    }
}
```

OUTPUT:



RESULT: The Service Registry and Discovery system was successfully implemented using Eureka.

- Microservices were registered dynamically.
- • Hard-coded service URLs were eliminated.
- • Service discovery enabled scalable and flexible microservice communication.

TASK 16: Configure an API Gateway to Route Requests to Microservices with Basic Load Balancing

Experiment / Implementation Document

Technology Stack	Spring Boot, Spring Cloud Gateway, Eureka Discovery Server,
Gateway Routing	Routes client requests to the appropriate microservice using
Load Balancing	Distributes requests across multiple service instances using lb:// service

Aim:

To configure an API Gateway that receives client requests, routes them to the correct microservice, and performs basic load balancing so that traffic is shared across multiple instances of the same service.

Algorithm:

1. Create a service registry (Eureka Server) so that all microservices and the API Gateway can register themselves.

2. Create the API Gateway application and add Spring Cloud Gateway, Eureka Client, and LoadBalancer dependencies.

3. Register the gateway with the discovery server.

4. Create one or more microservices such as product-service and run multiple instances of the same service on different ports.

5. Configure the gateway route using the logical service name with the prefix lb:// so that Spring Cloud LoadBalancer can choose an instance automatically.

6. When a client sends a request to the gateway, the gateway matches the path pattern and forwards the request to the target service.

7. The load balancer selects one instance from the available service instances and sends the request to it.

8. The selected microservice processes the request and returns the response to the gateway, which then returns it to the client.

9. Verify the setup by calling the same gateway endpoint multiple times and observing responses from different service instances.

Execution Flow Summary

Client Request	API Gateway	Microservice Instance
Sends /api/products/** request	Matches route and forwards using lb://	One of the available instances handles the

Code

The following sample uses Spring Boot and Spring Cloud. It demonstrates a discovery server, an API Gateway, and a product-service running on multiple instances.

1. Maven dependencies for API Gateway (pom.xml)

```
<dependencies>
   <dependency>
      <groupId>org.springframework.cloud</groupId>
      <artifactId>spring-cloud-starter-gateway</artifactId>
   </dependency>
   <dependency>
      <groupId>org.springframework.cloud</groupId>
      <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
   </dependency>
   <dependency>
      <groupId>org.springframework.cloud</groupId>
      <artifactId>spring-cloud-starter-loadbalancer</artifactId>
   </dependency>
   <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-actuator</artifactId>
   </dependency>
</dependices>

@SpringBootApplication
@EnableEurekaServer
public class DiscoveryServerApplication {
   public static void main(String[] args) {
      SpringApplication.run(DiscoveryServerApplication.class, args);
   }
}
server:
  port: 8761

spring:
  application:
    name: DISCOVERY-SERVER

eureka:
```

```
  client:
    register-with-eureka: false
    fetch-registry: false
@SpringBootApplication
public class ApiGatewayApplication {
    public static void main(String[] args) {
        SpringApplication.run(ApiGatewayApplication.class, args);
    }
}
server:
  port: 8080

spring:
  application:
    name: API-GATEWAY
  cloud:
    gateway:
      routes:
        - id: product-service-route
          uri: lb://PRODUCT-SERVICE
          predicates:
            - Path=/api/products/**
          filters:
            - StripPrefix=1

eureka:
  client:
    service-url:

# Instance 1
java -jar product-service.jar --server.port=8081
```

OUTPUT:

```
Started DiscoveryServerApplication on port 8761
Started ApiGatewayApplication on port 8080
Started ProductServiceApplication on port 8081
Started ProductServiceApplication on port 8082

GET /api/products/message
-> Routed to PRODUCT-SERVICE instance : 8081

GET /api/products/message
-> Routed to PRODUCT-SERVICE instance : 8082
```

RESULT:

Thus, the API Gateway was successfully configured to route client requests to the appropriate microservice. Basic load balancing was also implemented using the lb:// service identifier, allowing requests to be distributed across multiple running instances of the same service.

TASK-17

Inter-Service Communication Using REST APIs

Aim

To implement inter-service communication between microservices using REST APIs, enabling one service to call another dynamically using service names instead of hard-coded URLs.

Objective

- Understand communication between microservices
- Use REST APIs for interaction
- Implement service-to-service calls using `RestTemplate`
- Enable dynamic service discovery

Algorithm

1. Start
2. Create multiple microservices
3. Register services using Eureka Server
4. Enable service discovery in each service
5. Create REST endpoints in Service-2
6. Use `RestTemplate` in Service-1
7. Call Service-2 using service name
8. Return response
9. Test API
10. Stop

Program

1. Service-2 (Provider Service)

```
package com.example.service2;

import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api")
public class Service2Controller {

    @GetMapping("/message")
```

```
    public String getMessage() {
        return "Hello from Service-2";
    }
}
```

2. RestTemplate Configuration (Service-1)

```
package com.example.service1;

import org.springframework.context.annotation.Bean;
import
org.springframework.context.annotation.Configuration;
import org.springframework.web.client.RestTemplate;
import
org.springframework.cloud.client.loadbalancer.LoadBalanced;

@Configuration
public class AppConfig {

    @Bean
    @LoadBalanced
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}
```

3. Service-1 (Consumer Service)

```
package com.example.service1;

import
org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.client.RestTemplate;

@RestController
@RequestMapping("/consumer")
public class Service1Controller {

    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/call")
```

```
    public String callService() {
        return restTemplate.getForObject(
            "http://service-2/api/message",
            String.class
        );
    }
}
```

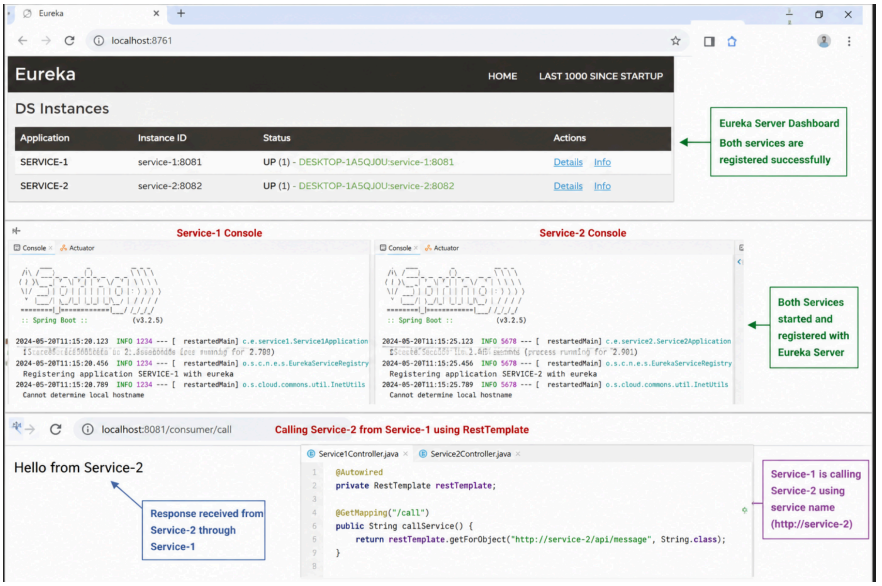
Application Properties

Service-1

```
spring.application.name=service-1
server.port=8081
eureka.client.service-url.defaultZone=http://
localhost:8761/eureka/
```

Service-2

```
spring.application.name=service-2
server.port=8082
eureka.client.service-url.defaultZone=http://loca
```



Result

- Service-1 successfully calls Service-2
- Communication achieved using REST APIs
- No hard-coded URLs used
- Dynamic service discovery implemented

TASK 18

AIM

To design and implement unit test cases for a Java-based microservice using a standard testing framework. The objective is to independently validate:

- Service-layer business logic
- REST controller endpoints
- Data access layer (repository interactions)

This ensures correctness, isolation, and reliability of each microservice component.

ALGORITHM

Step 1: Project Setup

1. Create a Maven-based Spring Boot project.
2. Add dependencies:
    - Spring Web
    - Spring Data JPA
    - H2 Database (for testing)
    - JUnit 5
    - Mockito

Step 2: Define Microservice Components

1. Create an **Entity class** (e.g., User)
2. Create a **Repository interface** extending JpaRepository
3. Create a **Service class** containing business logic
4. Create a **REST Controller** exposing endpoints

Step 3: Write Unit Tests for Service Layer

1. Mock repository using Mockito
2. Inject mocks into service
3. Test:
    - Valid data flow
    - Edge cases
    - Exception scenarios

Step 4: Write Unit Tests for Controller

1. Use @WebMvcTest
2. Mock service layer
3. Use MockMvc to simulate HTTP requests
4. Validate:
    - Response status
    - JSON structure
    - Returned data

Step 5: Test Data Handling

1. Use in-memory DB (H2) or mocked repository
2. Verify CRUD operations

3. Validate persistence behavior

Step 6: Execute Tests

1. Run using Maven:
2. Verify all test cases pass successfully

CODE

1. Entity Class

Java

```
@Entity
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
}
```

2. Repository

Java

```
public interface UserRepository extends JpaRepository<User, Long> {
}
```

3. Service Class

Java

```
@Service
public class UserService {
    @Autowired
    private UserRepository repo;
    public User saveUser(User user) {
        return repo.save(user);
    }
    public List<User> getAllUsers() {
        return repo.findAll();
    }
}
```

4. Controller

Java

```
@RestController
@RequestMapping("/users")
public class UserController {
    @Autowired
    private UserService service;
    @PostMapping
    public User addUser(@RequestBody User user) {
        return service.saveUser(user);
    }
    @GetMapping
    public List<User> getUsers() {
        return service.getAllUsers();
    }
}
```

UNIT TEST CASES

5. Service Layer Test

Java

```
@ExtendWith(MockitoExtension.class)
public class UserServiceTest {
    @Mock
    private UserRepository repo;
    @InjectMocks
    private UserService service;
    @Test
    void testSaveUser() {
        User user = new User();
        user.setName("John");
        Mockito.when(repo.save(user)).thenReturn(user);
        User saved = service.saveUser(user);
        assertEquals("John", saved.getName());
    }
}
```

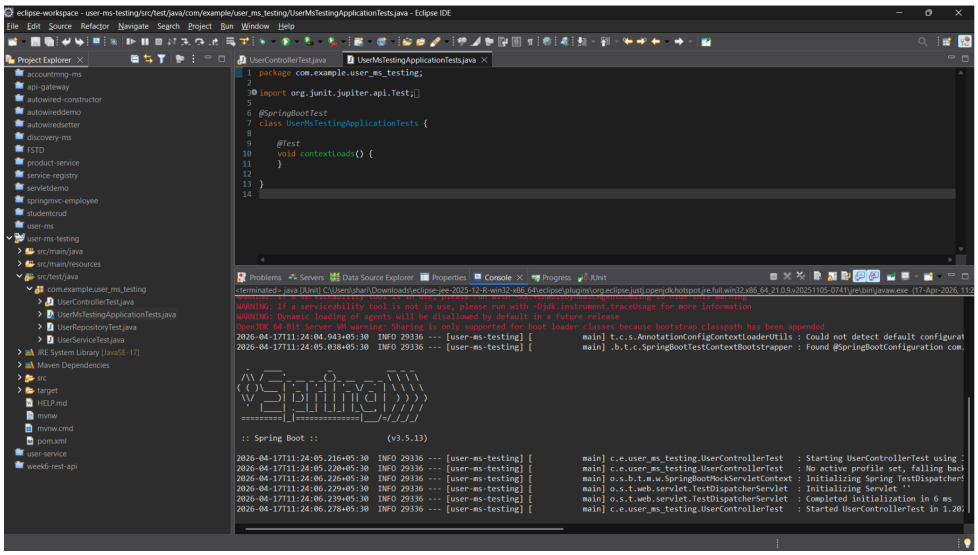
6. Controller Test

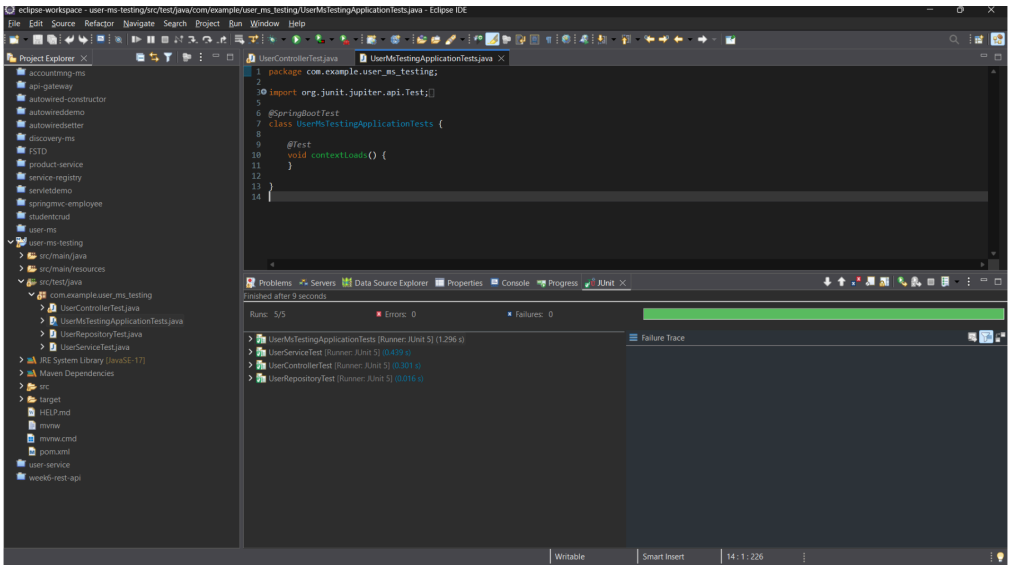
Java

```
@WebMvcTest(UserController.class)
public class UserControllerTest {
    @Autowired
    private MockMvc mockMvc;
```

```
    @MockBean
    private UserService service;
    @Test
    void testGetUsers() throws Exception {
        List<User> users = List.of(new User());
        Mockito.when(service.getAllUsers()).thenReturn(users);
        mockMvc.perform(get("/users"))
                .andExpect(status().isOk());
    }
}
```

OUTPUT:





RESULT: The unit testing of the Java microservice was successfully implemented using JUnit 5 and Mockito. Test cases were executed for the service layer, controller layer, and data handling ensuring application independently.

Task 19: Create a Git repository for a sample project and demonstrate basic Git operations such as initializing a repository, staging files, committing changes, and maintaining version history. Push the repository to GitHub and manage it using remote repositories.

Aim

To create a Git repository for a sample project and demonstrate:

- Repository initialization
- Staging files
- Committing changes
- Maintaining version history
- Connecting to remote repository
- Pushing project to GitHub

Tools Required

- Git
- GitHub
- VS Code / Terminal

Algorithm / Procedure

1. Create a project folder
2. Create HTML, CSS, JS files
3. Initialize Git repository
4. Add files to staging area
5. Commit files
6. Create repository in GitHub
7. Connect local repo to GitHub
8. Push files to remote repository
9. Verify output in GitHub

Program (Code)

index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Color Palette App</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
```

```
<div class="container">
  <h2>Color Palette App</h2>

  <input type="text" id="colorInput" placeholder="Enter color">

  <br><br>

  <button onclick="changeBackground()">Background</button>
  <button onclick="changeText()">Text Color</button>

  <p id="text">Welcome Students</p>
</div>

<script src="script.js"></script>

</body>
</html>
```

style.css

```
body {
  font-family: Arial;
  background-color: #f4f7fb;
  text-align: center;
}

.container {
  margin-top: 100px;
}

button {
  padding: 10px;
  margin: 5px;
  background-color: #2563eb;
  color: white;
  border: none;
}

#text {
  margin-top: 20px;
  font-size: 20px;
  background-color: #e0f2fe;
  padding: 10px;
}
```

script.js

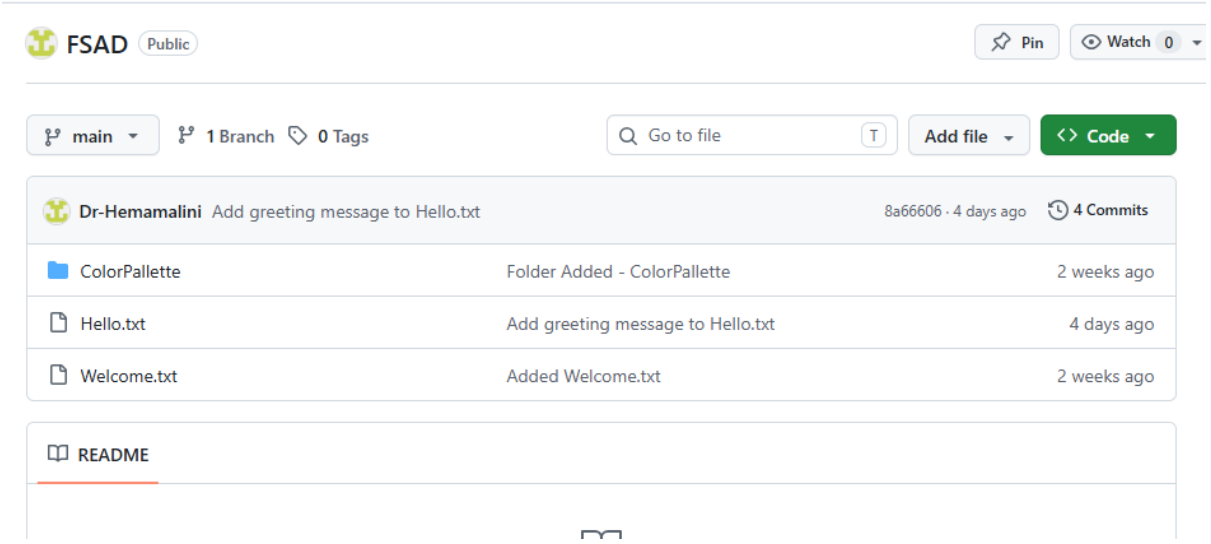
```
function changeBackground() {
  let color = document.getElementById("colorInput").value;
  document.getElementById("text").style.backgroundColor = color;
}
```

```
function changeText() {
  let color = document.getElementById("colorInput").value;
  document.getElementById("text").style.color = color;
}
```

Git Commands Used

```
cd D:\VSCode_Projects\Unit-IV
mkdir ColorPallette
cd ColorPallette

git init
git status
git add
git commit -m "Initial commit - Color Palette App"

git branch -M main
git remote add origin https://github.com/Dr-Hemamalini/FSAD.git
git pull origin main --allow-unrelated-histories
git push -u origin main
```

Output Screenshots



Git Output

- git init → repository created
- git status → files shown
- git add → staged files
- git commit → snapshot created
- git push → uploaded to GitHub

Result: Thus, a Git repository was successfully created for a sample project.
Basic Git operations such as initialization, staging, committing, and version history were performed.
The project was successfully pushed to GitHub using remote repository.

Task_20: Implement branching strategies in Git by creating feature branches. Perform merging and rebasing operations, and intentionally create merge conflicts to understand and resolve them effectively.

Git Branching Strategy

Aim

To create feature branches in Git, perform merge and rebase operations, and intentionally create merge conflicts to understand conflict resolution.

Scenario

A simple arithmetic calculator file is used.

Branches:

main
feature-add
feature-sub
feature-mul

Procedure

Step 1: Create Repository

```
mkdir git_branch_demo
cd git_branch_demo
git init
```

Step 2: Create Base File

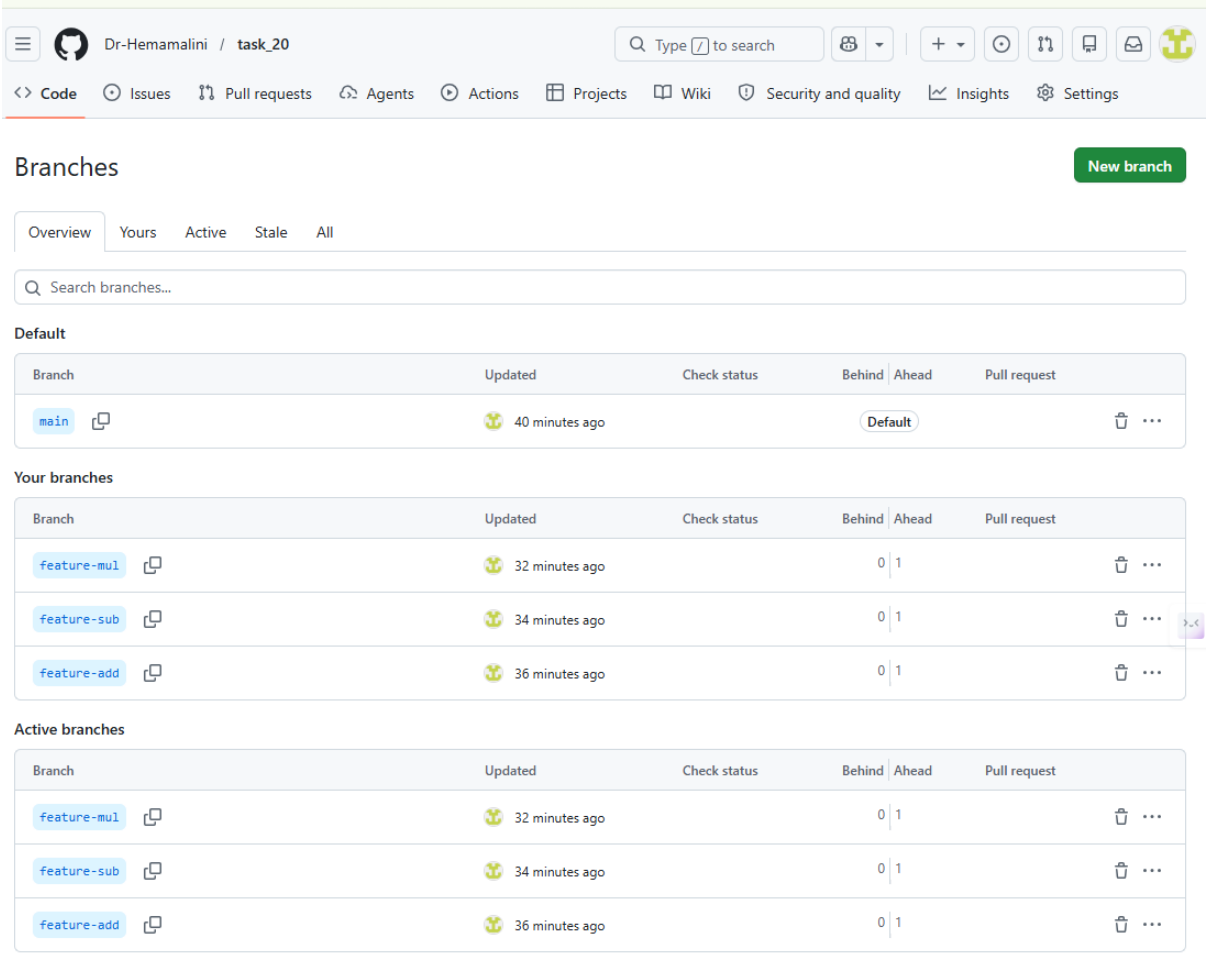
```
echo Basic Calculator > calc.txt
git add calc.txt
git commit -m "Initial commit"
```

Step 3: Create Addition Branch

```
git checkout -b feature-add
echo Addition: 10+5=15 >> calc.txt
git add calc.txt
git commit -m "Added addition feature"
```

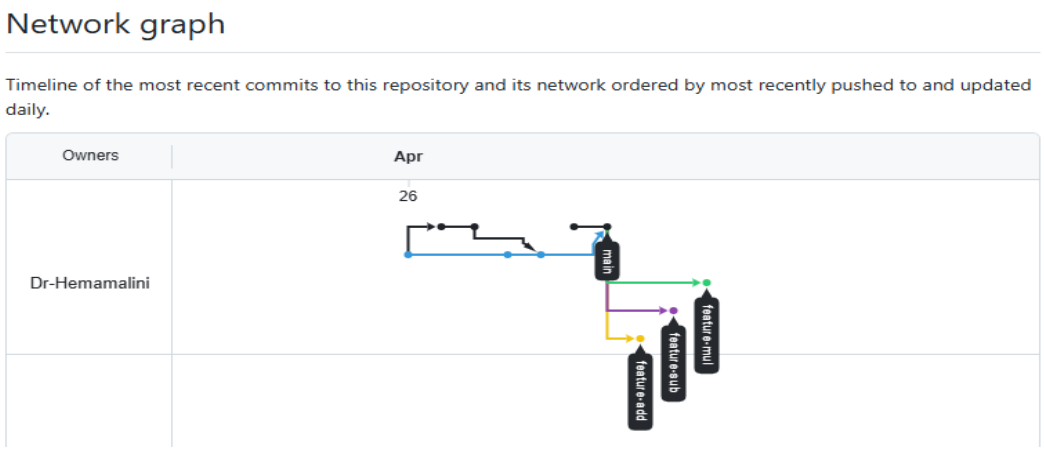
Step 4: Create Subtraction Branch

```
git checkout main
git checkout -b feature-sub
echo Subtraction: 10-5=5 >> calc.txt
git add calc.txt
git commit -m "Added subtraction feature"
```



Now Go to GitHub

Open your repo → Click: Insights → Network Graph



Step 5: Merge Addition Branch

git checkout main
git merge feature-add

Expected file:

Basic Calculator
Addition: 10+5=15

Step 6: Rebase Subtraction Branch

git checkout feature-sub
git rebase main

If conflict comes, edit calc.txt, then:

git add calc.txt
git rebase --continue

Step 7: Merge Subtraction Branch

git checkout main
git merge feature-sub

Expected file:

Basic Calculator
Addition: 10+5=15
Subtraction: 10-5=5

Create Merge Conflict

Step 8: Create Multiplication Branch

git checkout -b feature-mul
echo Result: Multiplication 10*5=50 > result.txt
git add result.txt
git commit -m "Added multiplication result"

Step 9: Modify Same File in Main

git checkout main
echo Result: Division 10/5=2 > result.txt
git add result.txt
git commit -m "Added division result"

Step 10: Merge and Create Conflict

git merge feature-mul

Expected conflict:

CONFLICT (content): Merge conflict in result.txt
Automatic merge failed

Step 11: Resolve Conflict

Open result.txt.

You may see:

<<<<<<< HEAD
Result: Division 10/5=2
=======

Result: Multiplication 10*5=50
>>>>>>> feature-mul

Replace with:

Result: Division 10/5=2
Result: Multiplication 10*5=50

Then commit:

git add result.txt
git commit -m "Resolved merge conflict"

Result

Thus, Git branching strategies were implemented successfully. Feature branches were created, merge and rebase operations were performed, and merge conflicts were intentionally created and resolved manually.

Task 21: Implement a CI/CD pipeline using Jenkins, GitHub Actions, or GitLab CI/CD. Automate code checkout, build, test, and deployment steps, and demonstrate continuous integration and continuous deployment for the application

Implementation of CI/CD Pipeline using Jenkins

Aim

To implement a Continuous Integration and Continuous Deployment (CI/CD) pipeline using Jenkins to automate code checkout, build, testing, and deployment of a Node.js application.

Tools Required

- Jenkins
- GitHub
- Node.js
- Git
- VS Code

Project Description

A simple Node.js application is created to perform arithmetic addition and validate it using a test script.

Procedure

Step 1: Create Project

mkdir jenkins-cicd-demo
cd jenkins-cicd-demo
npm init -y

Step 2: Create Application File (app.js)

```
function add(a, b) {
  return a + b;
```

```
}

console.log("Application running successfully");
console.log("Addition:", add(5, 3));

module.exports = add;
```

Step 3: Create Test File (app.test.js)

```
const add = require("./app");

if (add(5, 3) === 8) {
  console.log("Test Passed");
  process.exit(0);
} else {
  console.log("Test Failed");
  process.exit(1);
}
```

Step 4: Update package.json

```
{
  "name": "jenkins-cicd-demo",
  "version": "1.0.0",
  "main": "app.js",
  "scripts": {
    "test": "node app.test.js",
    "build": "echo Build completed successfully"
  }
}
```

Step 5: Push Code to GitHub

```
git init
git add .
git commit -m "Initial commit"
git branch -M main
git remote add origin https://github.com/Dr-Hemamalini/cicd-demo.git
git push -u origin main
```

Step 6: Create GitHub Token

1. Go to GitHub → Settings → Developer Settings
2. Select **Personal Access Token**
3. Generate token with **repo access**
4. Copy the token

Step 7: Configure Credentials in Jenkins

1. Open Jenkins Dashboard
2. Go to **Manage Jenkins → Manage Credentials**
3. Add credentials:
    - Kind: Username & Password
    - Username: GitHub username

- Password: Personal Access Token
- ID: github-token

Step 8: Create Jenkins Pipeline Job

1. Click **New Item**
2. Enter name: CICD-Demo
3. Select **Pipeline** → Click OK

Step 9: Create Jenkinsfile

```
pipeline {
  agent any

  stages {
    stage('Checkout Code') {
      steps {
        checkout([
          $class: 'GitSCM',
          branches: [[name: '*/main']],
          userRemoteConfigs: [[
            url: 'https://github.com/Dr-Hemamalini/cicd-demo.git',
            credentialsId: 'github-token'
          ]]
        ])
      }
    }

    stage('Install Dependencies') {
      steps {
        bat 'npm install'
      }
    }

    stage('Build Application') {
      steps {
        bat 'npm run build'
      }
    }

    stage('Run Tests') {
      steps {
        bat 'npm test'
      }
    }

    stage('Deploy Application') {
      steps {
        bat 'echo Application deployed successfully'
      }
    }
  }
}
```

Step 10: Push Jenkinsfile

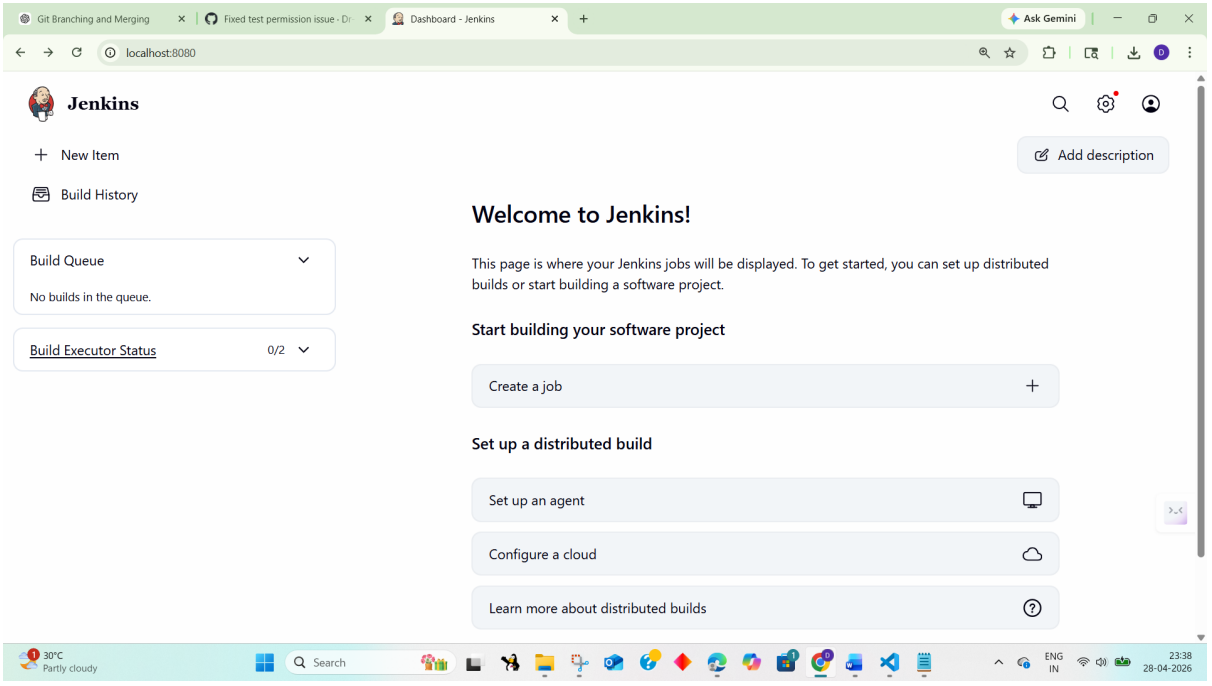
git add Jenkinsfile
git commit -m "Added Jenkins pipeline"
git push

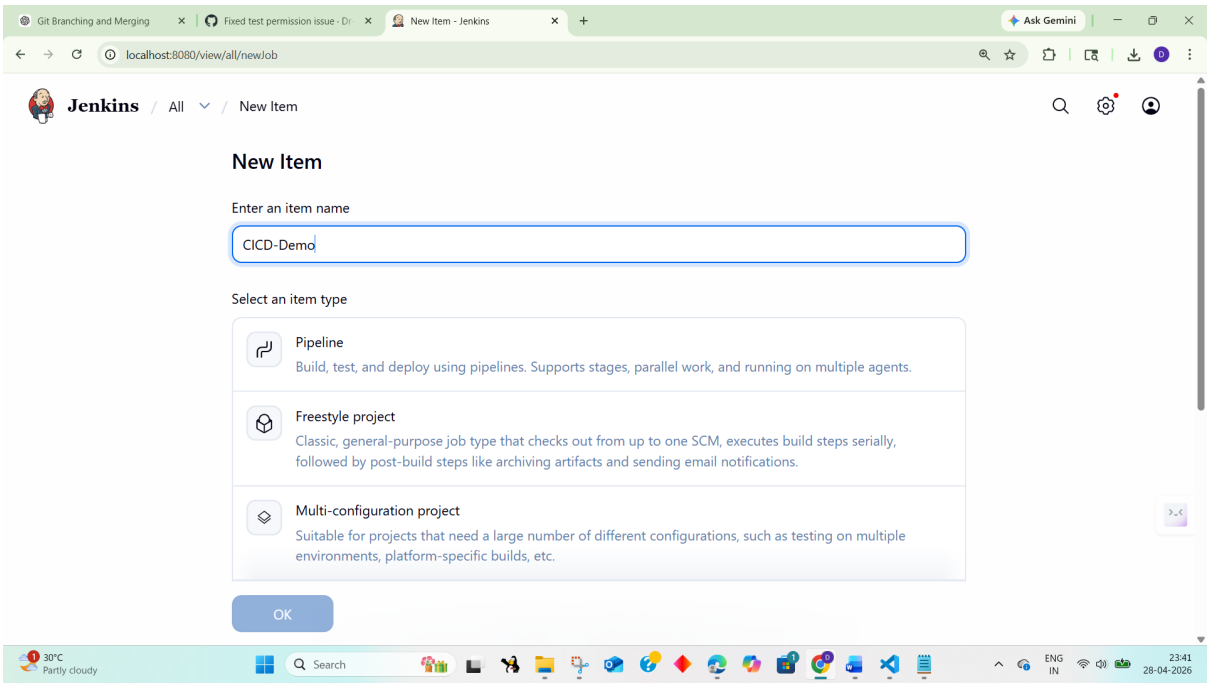
Step 11: Run Pipeline

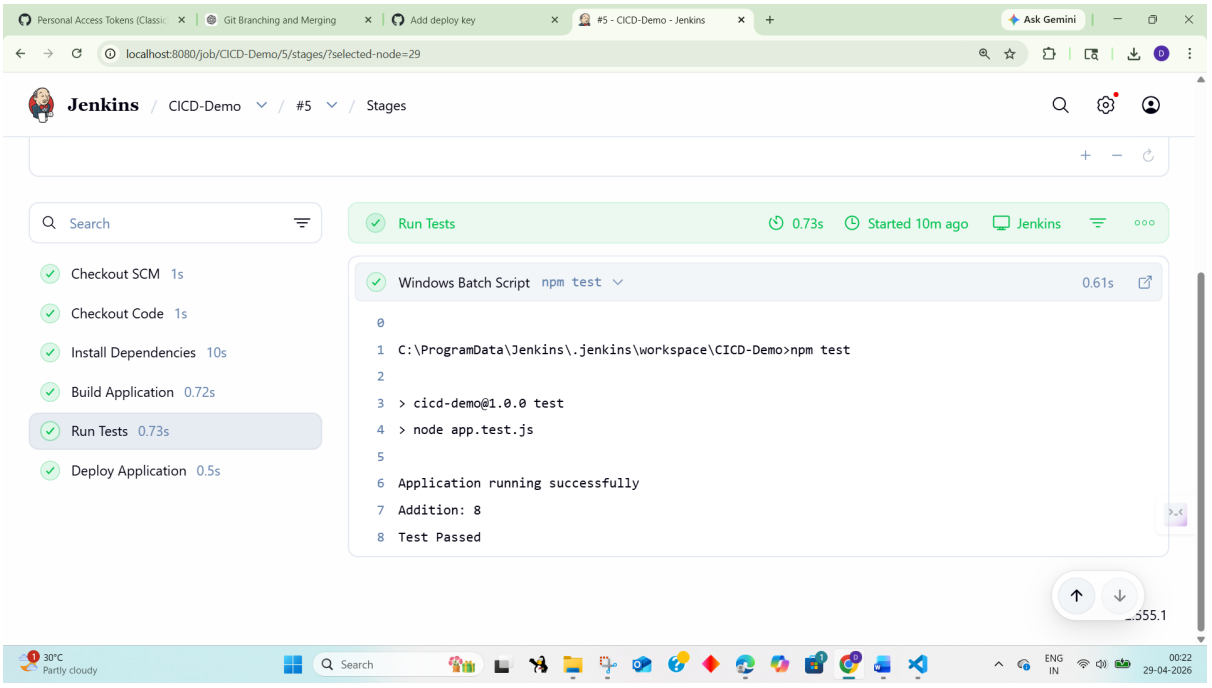
1. Open Jenkins Job
2. Click **Build Now**
3. View Console Output

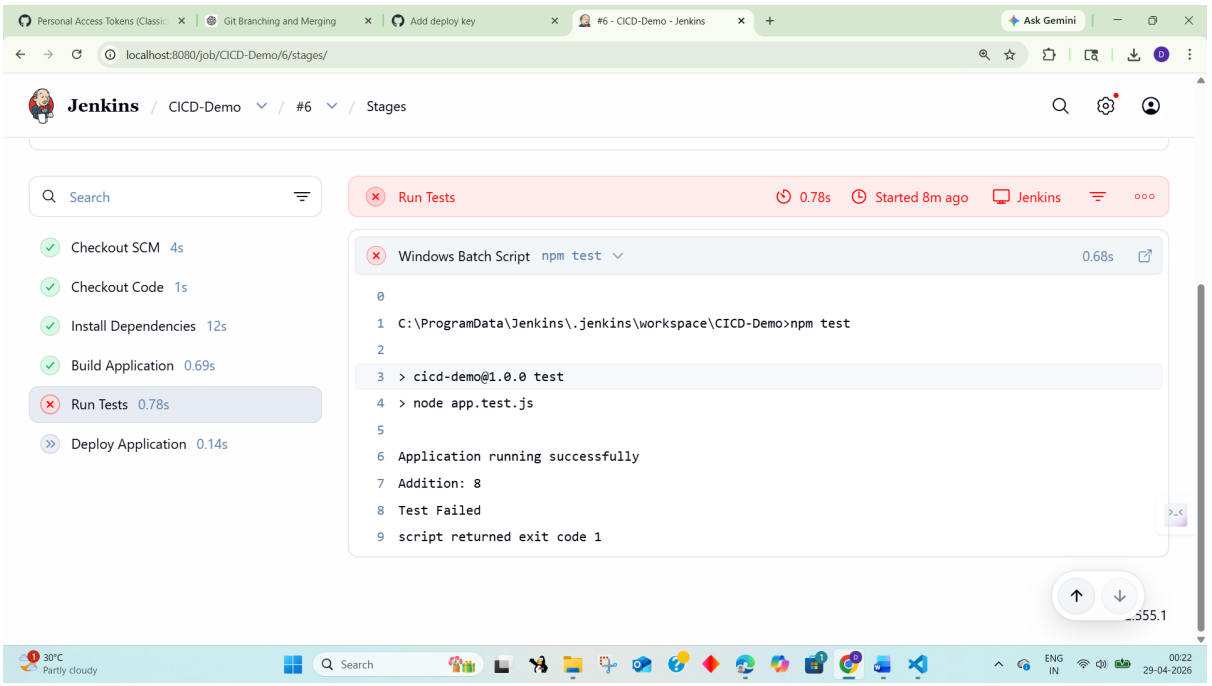
Expected Output

Checkout Code → Success
Install Dependencies → Success
Build → Build completed successfully
Test → Test Passed
Deploy → Application deployed successfully




Git Branching and Merging
Fixed test permission issue - Dr
New Item - Jenkins
Ask Gemini
localhost:8080/view/all/newJob
Jenkins / All / New Item
New Item
Enter an item name
CICD-Demo
Select an item type
Pipeline
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
Multi-configuration project
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
OK
30°C
Partly cloudy
Search
ENG
IN
23:41
28-04-2026


Personal Access Tokens (Classic
Git Branching and Merging
Add deploy key
#5 - CICD-Demo - Jenkins
Ask Gemini
localhost:8080/job/CICD-Demo/5/stages/?selected-node=29
Jenkins / CICD-Demo / #5 / Stages
Search
Checkout SCM 1s
Checkout Code 1s
Install Dependencies 10s
Build Application 0.72s
Run Tests 0.73s
Deploy Application 0.5s
Run Tests
0.73s
Started 10m ago
Jenkins
Windows Batch Script npm test
0.61s
0
1 C:\ProgramData\Jenkins\.jenkins\workspace\CICD-Demo>npm test
2
3 > cicd-demo@1.0.0 test
4 > node app.test.js
5
6 Application running successfully
7 Addition: 8
8 Test Passed
30°C
Partly cloudy
Search
ENG
IN
00:22
29-04-2026



Result

Thus, a CI/CD pipeline was successfully implemented using Jenkins. The pipeline automated code checkout, dependency installation, build process, testing, and deployment of the Node.js application.

Task 22: Explore major cloud service providers such as AWS, Google Cloud, and Microsoft Azure. Identify and compare their core services related to compute, storage, databases, and networking, along with the pay-as-you-use pricing model.

COMPARISON OF CLOUD SERVICES: AWS, GOOGLEC CLOUD, MICROSOFT AZURE

AIM: To develop a cloud comparison application that displays and compares services and pricing models of AWS, GCP, and Azure

ALGORITHM:

1. Start the program
2. Create project files (HTML, CSS, JS, server.js, db.js)
3. Design user interface with dropdown for cloud providers
4. Initialize cloud service data (AWS, GCP, Azure) in database module
5. Capture selected provider using JavaScript
6. Send request to server to fetch service details
7. Display compute, storage, database, and networking services
8. Accept user input for compute hours and storage usage
9. Send input data to server for cost calculation
10. Calculate cost based on pay-as-you-use pricing model
11. Display total cost dynamically on UI
12. Stop the program

PROGRAM:

INDEX.HTML:

```
<!DOCTYPE html>
<html>
<head>
<title>Cloud Comparison</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
<div class="container">
<h2>Cloud Service Comparison</h2>
<select id="provider" onchange="loadData()">
<option value="aws">AWS</option>
<option value="gcp">GCP</option>
<option value="azure">Azure</option>
</select>
<h3>Services</h3>
<ul id="services"></ul>
<h3>Pricing Calculator</h3>
<input id="hours" placeholder="Compute Hours">
<input id="storage" placeholder="Storage (GB)">
<button onclick="calculate()">Calculate Cost</button>
<h3 id="result"></h3>
</div>
<script src="script.js"></script>
</body>
</html>
```

STYLE.CSS:

```
body {
font-family: Arial;
background: #1e1e2f; color: white; display:flex;
justify-content:center; align-items:center;
height:100vh;
}
.container {
```

```
text-align:center; width: 350px;
}
input, select, button { margin: 5px; padding: 8px;
}
li {
background: #333; margin: 5px; padding: 5px;
}
```

SCRIPT.JS:

```
function loadData() {
const provider = document.getElementById("provider").value; fetch(`/services/${provider}`)
.then(res => res.json())
.then(data => {
let list = document.getElementById("services"); list.innerHTML = "";
Object.entries(data).forEach(([key, value]) => { let li = document.createElement("li");
li.innerText = `${key}: ${value}`; list.appendChild(li);
});
});
}
function calculate() {
const provider = document.getElementById("provider").value; const hours =
document.getElementById("hours").value; const storage = document.getElementById("storage").value;
fetch("/cost", {
method: "POST",
headers: {"Content-Type":"application/json"}, body: JSON.stringify({ provider, hours, storage })
})
.then(res => res.json())
.then(data => {
document.getElementById("result").innerText = "Cost: $" + data.cost;
});
}
loadData();
```

SERVER.JS:

```
const express = require("express"); const db = require("./db");
const app = express(); app.use(express.json());
app.use(express.static(_dirname)); app.get("/services/:provider", (req, res) => {
res.json(db.getServices(req.params.provider));
```

});

app.post("/cost", (req, res) => {

const { provider, hours, storage } = req.body;

const cost = db.calculate(provider, hours, storage); res.json({ cost });

});

app.listen(3000, () => {

console.log("http://localhost:3000");

});

DB.JS:

const services = { aws: {

Compute: "EC2", Storage: "S3", Database: "RDS", Network: "VPC"

},

gcp: {

Compute: "Compute Engine", Storage: "Cloud Storage", Database: "Cloud SQL", Network: "VPC"

},

azure: {

Compute: "Virtual Machines", Storage: "Blob Storage",

Database: "SQL Database", Network: "VNet"

}

};

const pricing = {

aws: { compute: 0.1, storage: 0.02 }, gcp: { compute: 0.09, storage: 0.025 }, azure: { compute: 0.11, storage: 0.018 }

};

function getServices(provider) { return services[provider];

}

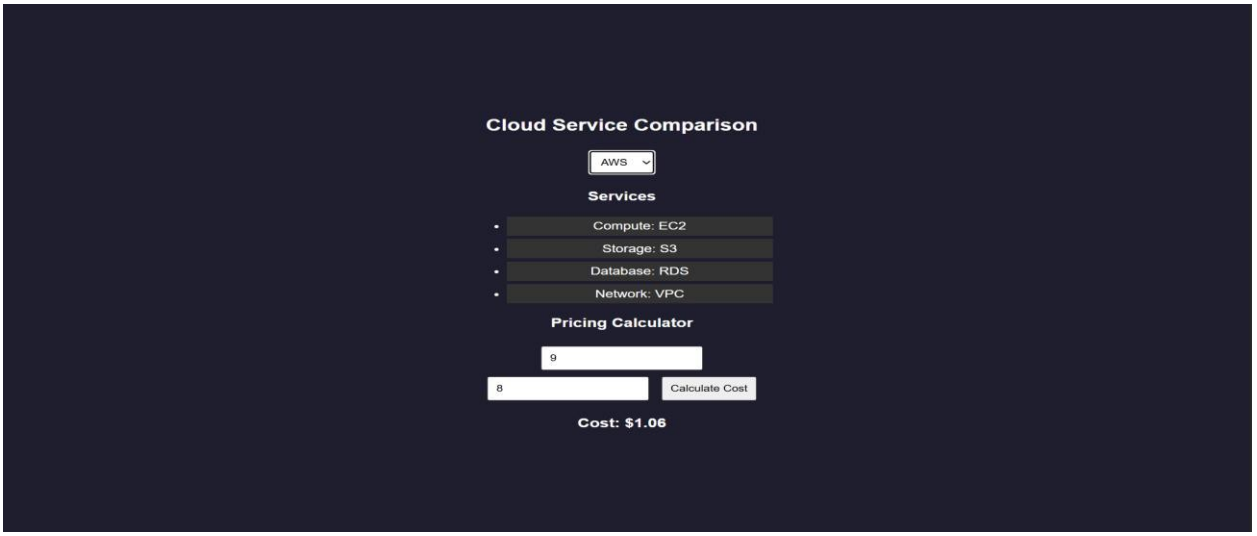
function calculate(provider, hours, storage) { const p = pricing[provider];

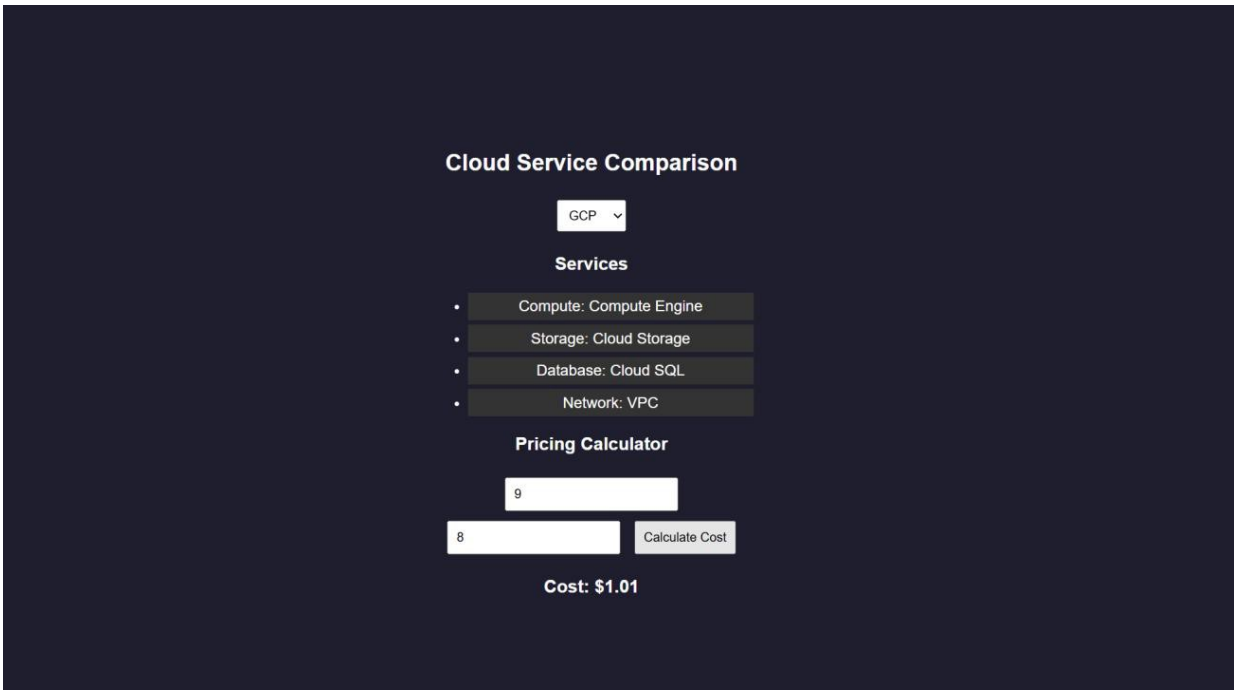
return (hours * p.compute + storage * p.storage).toFixed(2);

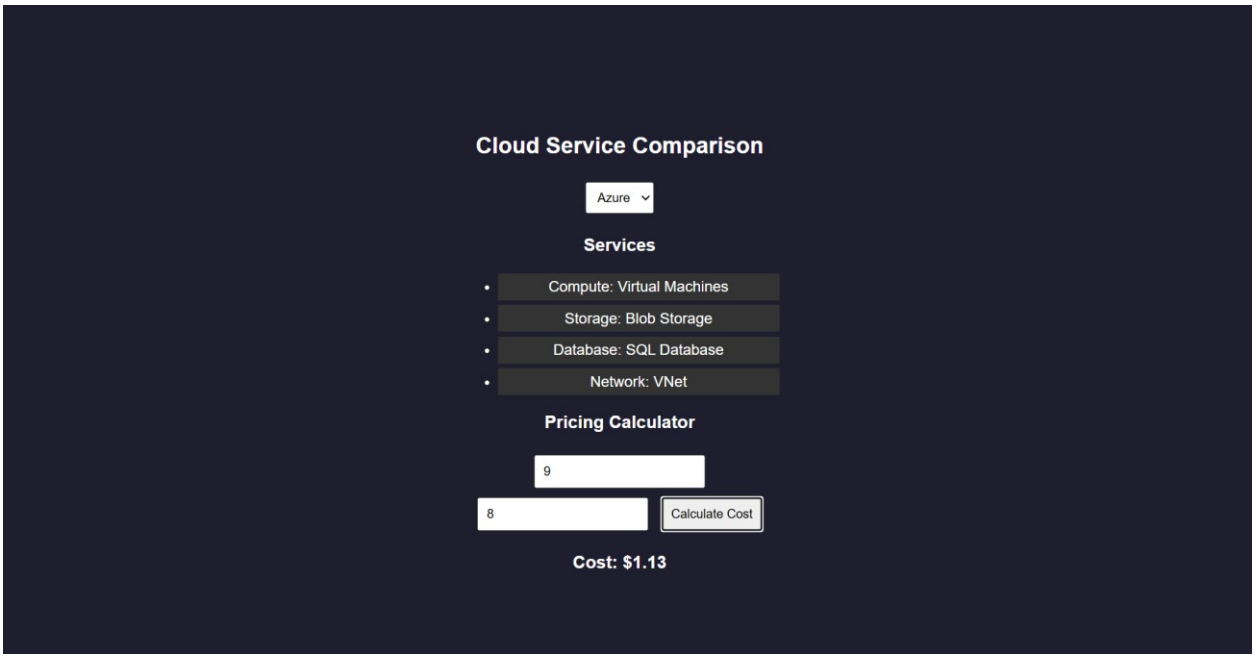
}

module.exports = { getServices, calculate };

OUTPUT:







RESULTS: A cloud comparison application was successfully developed to display services and simulate pay-as-you-use pricing across AWS, GCP, and Azure.

Task 23: Apply Vibe Coding and Prompt Engineering techniques using a Generative AI tool. Design effective prompts to generate code snippets, cloud configuration templates, or application logic, and evaluate the quality and efficiency of the generated outputs for real-world business applications.

AI PROMPT APP USING VIBE CODING

AIM: To develop an application that demonstrates prompt engineering by generating and evaluating code or cloud configurations based on user input.

ALGORITHM:

1. Start the program
2. Create project files (HTML, CSS, JS, server.js, db.js)
3. Design UI with input field for user prompt
4. Capture user prompt using JavaScript
5. Send prompt to server using fetch API
6. Receive prompt data in backend (server.js)
7. Process prompt using predefined logic (db.js)
8. Generate output (code / cloud config / response)
9. Evaluate output quality and assign score
10. Send generated output and score to frontend
11. Display output and score on UI
12. Stop the program

PROGRAM:

INDEX.HTML:

```
<!DOCTYPE html>

<html>

<head>

<title>AI Prompt App</title>

<link rel="stylesheet" href="style.css">

</head>

<body>
```

```
<div class="container">
<h2>Prompt Engineering Tool</h2>
<textarea id="prompt" placeholder="Enter your prompt..."></textarea>
<button onclick="generate()">Generate Output</button>
<h3>Generated Output</h3>
<pre id="output"></pre>
<h3>Quality Score</h3>
<p id="score"></p>
</div>
<script src="script.js"></script>
</body>
</html>
```

STYLE.CSS:

```
body {
font-family: Arial;
background: #202038; color: white;
display:flex;
justify-content:center; align-items:center;
height:100vh;
}
.container { width: 400px;
text-align: center;
}
textarea { width: 100%; height: 100px; margin: 5px;
}
button { padding: 10px; margin: 10px;
background: #00c6ff; border: none;
color: white;
}
pre {
background: #333; padding: 10px;
}
```

SCRIPT.JS:

```
function generate() {
const prompt = document.getElementById("prompt").value; fetch("/generate", {
```

```
method: "POST",
headers: {"Content-Type":"application/json"}, body: JSON.stringify({ prompt })
})
.then(res => res.json())
.then(data => {
document.getElementById("output").innerText = data.output;
document.getElementById("score").innerText = data.score + "/10";
});
}
```

SERVER.JS:

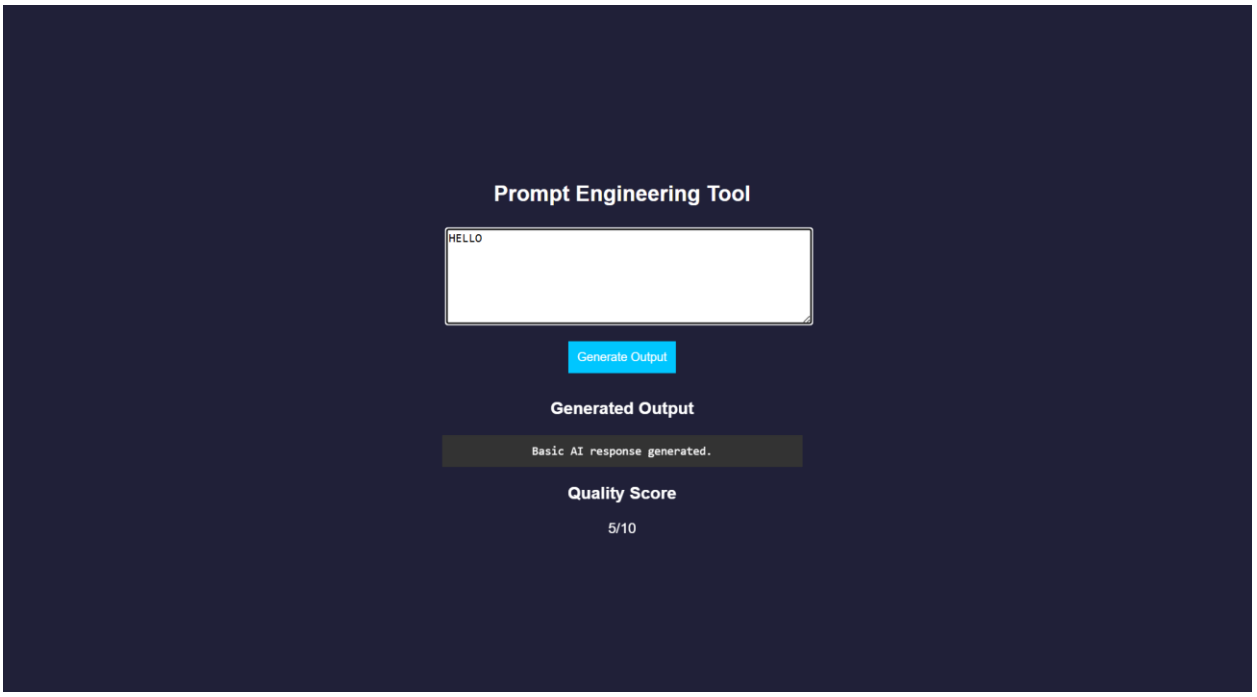
```
const express = require("express"); const db = require("./db");
const app = express();
// Middleware
app.use(express.json());
app.use(express.static(_dirname)); // serve HTML
// Home route (optional but useful) app.get("/", (req, res) => {
res.sendFile(_dirname + "/index.html");
});
// Generate output app.post("/generate", (req, res) => {
const result = db.processPrompt(req.body.prompt); res.json(result);
});
// START SERVER WITH LINK
app.listen(3000, () => {
console.log(" •  Server running at:");
console.log(" http://localhost:3000");
});
```

DB.JS:

```
function processPrompt(prompt) { let output = "";
let score = 5;
if (prompt.toLowerCase().includes("code")) { output = `function greet() {
console.log("Hello from AI!");
}`;
```

```
score = 9;
}
else if (prompt.toLowerCase().includes("cloud")) { output = `AWS EC2 Config:
    - Instance: t2.micro
    - Region: us-east-1`; score = 8;
}
else {
output = "Basic AI response generated."; score = 5;
}
return { output, score };
}
module.exports = { processPrompt };
```

OUTPUT:



RESULT: A prompt engineering application was successfully developed to generate and evaluate AI-based outputs for code and cloud configurations.

Task 24: Use Vibe Coding to build a complete cloud-based feature using Generative AI. Write iterative prompts to generate a REST API, cloud deployment steps, and security configurations. Refine the prompts based on the output quality, document prompt versions, and evaluate how prompt engineering improves code accuracy, scalability, and real-world usability.

Vibe Cloud App

AIM: To develop an application that uses iterative prompt engineering to generate and evaluate REST APIs, cloud deployment steps, and security configurations.

ALGORITHM:

1. Start the program
2. Create project files (HTML, CSS, JS, server.js, db.js)
3. Design UI to accept user prompts
4. Capture prompt input using JavaScript
5. Send prompt to backend using fetch API
6. Receive prompt in server and forward to processing module
7. Store prompt in history for version tracking
8. Analyze prompt type (REST API / Cloud / Security)
9. Generate corresponding output based on prompt
10. Evaluate output quality and assign score
11. Return output and score to frontend and display results
12. Stop the program

PROGRAM:

INDEX.HTML:

```
<!DOCTYPE html>

<html>

<head>

<title>Vibe Coding App</title>

<link rel="stylesheet" href="style.css">

</head>

<body>

<div class="container">

<h2>Vibe Coding Cloud Generator</h2>
```

```
<textarea id="prompt" placeholder="Enter prompt..."></textarea>
<button onclick="generate()">Generate</button>
<h3>Output</h3>
<pre id="output"></pre>
<h3>Score</h3>
<p id="score"></p>
<h3>Prompt Versions</h3>
<ul id="history"></ul>
</div>
<script src="script.js"></script>
</body>
</html>
```

STYLE.CSS:

```
body {
font-family: Arial;
background: #b98bdb; color: white;
display:flex;
justify-content:center; align-items:center;
height:100vh;
}
.container { width: 400px;
text-align: center;
}
textarea { width: 100%; height: 100px;
}
button { padding: 10px; margin: 10px;
background: #00c6ff; border: none;
}
pre {
background: #333; padding: 10px;
}
li {
background: #444; margin: 5px; padding: 5px;
}
```

SCRIPT.JS:

```
function generate() {
const prompt = document.getElementById("prompt").value; fetch("/generate", {
method: "POST",
headers: {"Content-Type":"application/json"},
body: JSON.stringify({ prompt })
})
.then(res => res.json())
.then(data => {
document.getElementById("output").innerText = data.output;
document.getElementById("score").innerText = data.score + "/10"; loadHistory();
});
}
function loadHistory() { fetch("/history")
.then(res => res.json())
.then(data => {
let list = document.getElementById("history"); list.innerHTML = "";
data.forEach((p, i) => {
let li = document.createElement("li"); li.innerText = `v${i+1}: ${p.prompt}`; list.appendChild(li);
});
});
}
loadHistory();
```

SERVER.JS:

```
const express = require("express"); const db = require("./db");
```

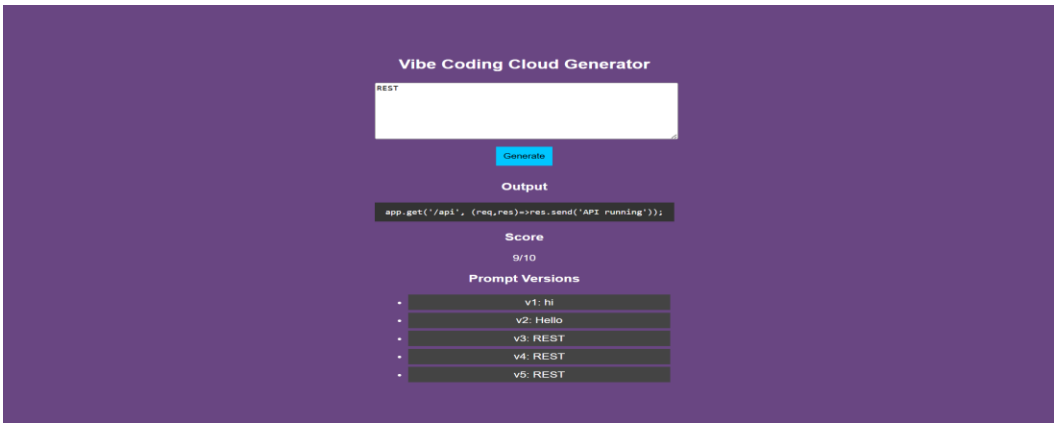
```
const app = express(); app.use(express.json());
app.use(express.static(_dirname)); app.post("/generate", (req, res) => {
const result = db.process(req.body.prompt); res.json(result);
});
app.get("/history", (req, res) => { res.json(db.getHistory());
});
app.listen(3000, () => {
console.log(" • http://localhost:3000");
});
```

DB.JS:

```
let history = [];
function process(prompt) { let output = "";
let score = 5;
history.push({ prompt });
if (prompt.includes("REST")) {
output = `app.get('/api', (req,res)=>res.send('API running'));`; score = 7;
}
else if (prompt.includes("cloud")) { output = `Deploy using AWS EC2:
1. Launch instance
2. Install Node.js
3. Run app`;
score = 8;
}
else if (prompt.includes("security")) {
output = `Use HTTPS, JWT authentication, and firewall rules`; score = 9;
}
score += Math.min(history.length, 2); return { output, score };
}
function getHistory() { return history;
}
module.exports = { process, getHistory };
```

OUTPUT:

RESULT: A Vibe Coding application was successfully developed to generate and refine cloud-based features using iterative prompt engineering.


Vibe Coding Cloud Generator
REST
Generate
Output
app.get('/api', (req,res)=>res.send('API running'));
Score
9/10
Prompt Versions
v1: hi
v2: Hello
v3: REST
v4: REST
v5: REST